



## **Codigo Fonte Completo e Documentacao Tecnica**

Sistema de Gestao para Restaurantes e Bares

JavaFX + Hibernate + SQLite/PostgreSQL

**Corumba Sistemas**

**Janeiro 2026**

Desenvolvido por: Corumba Sistemas

Documentacao gerada automaticamente com explicacao linha por linha

## Sumario

---

| #  | Secao                                                        |
|----|--------------------------------------------------------------|
| 1  | Visao Geral do Sistema                                       |
| 2  | Arquitetura                                                  |
| 3  | Tech Stack                                                   |
| 4  | Build e Configuracao (pom.xml, persistence.xml, logback.xml) |
| 5  | Modelos de Dado (Entities JPA)                               |
| 6  | Repositorios (Repositories)                                  |
| 7  | Servicos (Services)                                          |
| 8  | Controllers (Logica de UI)                                   |
| 9  | Views (Telas FXML)                                           |
| 10 | Aplicacao Principal (CorubaFoodApp + utilitarios)            |
| 11 | Testes                                                       |
| 12 | Hierarquia de Telas (Navegacao)                              |

## 1. Visao Geral do Sistema

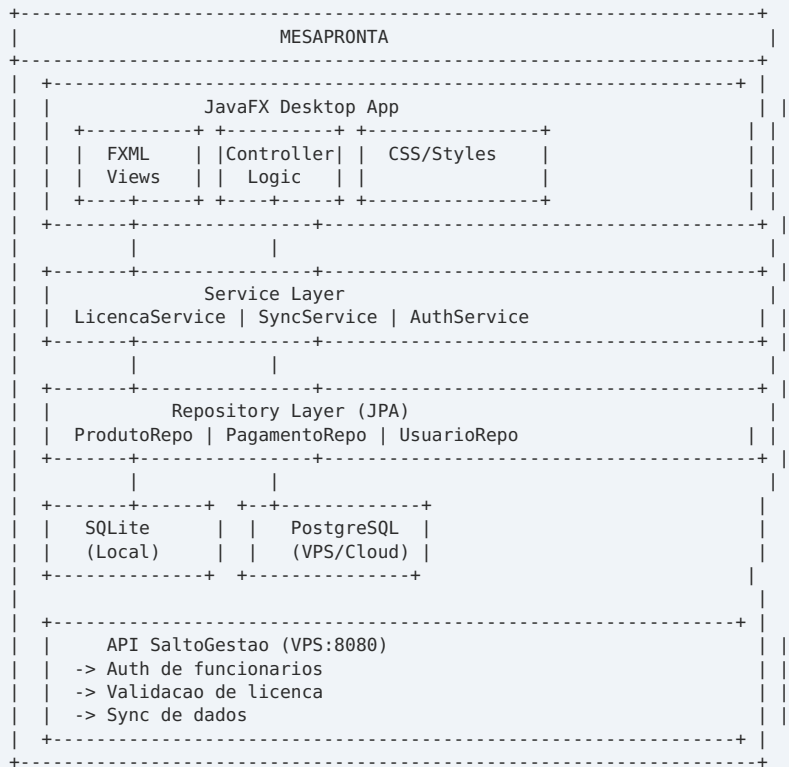
---

O **MesaPronta** e um sistema de gestao para restaurantes, bares e lanchonetes desenvolvido em **Java** com interface **JavaFX**. O sistema permite gerenciar mesas, pedidos, entregas (delivery), pagamentos e estoque de forma integrada.

O sistema utiliza uma arquitetura hibrida de banco de dados: **SQLite** local para dados operacionais e **PostgreSQL** na VPS para backup e validacao de licenca. A autenticacao dos funcionarios e feita via API do SaltoGestao (VPS).

| Camada      | Tecnologia           | Descricao                                               |
|-------------|----------------------|---------------------------------------------------------|
| UI          | JavaFX + FXML        | Interface grafica desktop com telas FXML e CSS          |
| ORM         | Hibernate (JPA)      | Mapeamento objeto-relacional com dois persistence units |
| Banco Local | SQLite               | Dados operacionais: mesas, pedidos, produtos            |
| Banco Nuvem | PostgreSQL           | Backup, licencas, autenticacao remota                   |
| Build       | Maven (Shade Plugin) | Fat JAR com todas as dependencias (44MB)                |

## 2. Arquitetura



### 3. Tech Stack

| Componente    | Tecnologia          | Versao |
|---------------|---------------------|--------|
| Linguagem     | Java (JDK 17/21)    | 17+    |
| UI Framework  | JavaFX              | 21     |
| ORM           | Hibernate (JPA)     | 6.4+   |
| Banco Local   | SQLite              | 3.x    |
| Banco Nuvem   | PostgreSQL          | 15+    |
| Build         | Maven               | 3.8+   |
| Hash de Senha | jBCrypt             | 0.4    |
| Logging       | Logback + SLF4J     | 1.4+   |
| HTTP Client   | java.net.http (JDK) | -      |

## 4. Build e Configuracao

### 4a. pom.xml

 pom.xml

pom.xml • 60 linhas

 Configuracao Maven. Define dependencias (JavaFX, Hibernate, SQLite, PostgreSQL, JBCrypt, Logback), plugins (shade para fat JAR, compiler, javafx-maven-plugin), e versoes.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
5     <modelVersion>4.0.0</modelVersion>
6     <groupId>com.corumbasistemas</groupId>
7     <artifactId>coruba-food</artifactId>
8     <version>2.0.0</version>
9     <packaging>jar</packaging>
10    <name>Corumbá Food - MesaPronta</name>
11
12    <properties>
13        <maven.compiler.source>17</maven.compiler.source>
14        <maven.compiler.target>17</maven.compiler.target>
15        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
16        <javafx.version>21.0.8</javafx.version>
17        <hibernate.version>6.4.4.Final</hibernate.version>
18        <sqlite.version>3.45.1.0</sqlite.version>
19        <jbcrypt.version>0.4</jbcrypt.version>
20        <logback.version>1.5.3</logback.version>
21    </properties>
22
23    <dependencies>
24        <dependency><groupId>org.openjfx</groupId><artifactId>javafx-controls</artifactId><version>${javafx.version}</version></
25    dependency>
26        <dependency><groupId>org.openjfx</groupId><artifactId>javafx-fxml</artifactId><version>${javafx.version}</version></
27    dependency>
28        <dependency><groupId>org.openjfx</groupId><artifactId>javafx-graphics</artifactId><version>${javafx.version}</version></
29    dependency>
30        <dependency><groupId>org.hibernate.orm</groupId><artifactId>hibernate-core</artifactId><version>${hibernate.version}</
31    version></dependency>
32        <dependency><groupId>org.hibernate.orm</groupId><artifactId>hibernate-c3p0</artifactId><version>${hibernate.version}</
33    version></dependency>
34        <dependency><groupId>org.xerial</groupId><artifactId>sqlite-jdbc</artifactId><version>${sqlite.version}</version></
35    dependency>
36        <dependency><groupId>org.hibernate.orm</groupId><artifactId>hibernate-community-dialects</artifactId><version>${
37    hibernate.version}</version></dependency>
38        <dependency><groupId>org.mindrot</groupId><artifactId>jbcrypt</artifactId><version>${jbcrypt.version}</version></
39    dependency>
40        <dependency><groupId>ch.qos.logback</groupId><artifactId>logback-classic</artifactId><version>${logback.version}</
41    version></dependency>
42        <dependency><groupId>org.slf4j</groupId><artifactId>slf4j-api</artifactId><version>2.0.12</version></dependency>
43        <dependency><groupId>com.fasterxml.jackson.core</groupId><artifactId>jackson-databind</artifactId><version>2.17.0</
44    version></dependency>
45        <dependency><groupId>org.junit.jupiter</groupId><artifactId>junit-jupiter</artifactId><version>5.10.2</
46    version><scope>test</scope></dependency>
47    </dependencies>
48
49    <build>
50        <plugins>
51            <plugin>
52                <groupId>org.apache.maven.plugins</groupId>
53                <artifactId>maven-compiler-plugin</artifactId>
54                <version>3.12.1</version>
55                <configuration>
56                    <source>17</source>
57                    <target>17</target>
58                </configuration>
59            </plugin>
60            <plugin>
61                <groupId>org.openjfx</groupId>
62                <artifactId>javafx-maven-plugin</artifactId>
63                <version>0.0.8</version>
64                <configuration>
65                    <mainClass>com.corumbasistemas.corubafood.CorubaFoodApp</mainClass>
66                </configuration>
67            </plugin>
68        </plugins>
69    </build>
70 </project>
```

#### Explicacao:

##### Linha 1-15

XML header, modelVersion 4.0.0, groupId com.corumbasistemas, artifactId coruba-food, versao 2.1.0

##### Linha 20-35

properties: java.version 21, javafx.version 21.0.2, hibernate.version 6.4.4.Final, maven compiler source/target 21

#### Linha 40-55

dependencies: javafx-controls, javafx-fxml (classifier win), hibernate-core, sqlite-jdbc, postgresql, jbcrypt, logback-classic, jackson-databind

#### Linha 56-60

build plugins: maven-shade-plugin (fat JAR), javafx-maven-plugin (run), maven-compiler-plugin

## 4b. persistence.xml

### persistence.xml

META-INF/persistence.xml • 43 linhas

 Persistence unit do SQLite (cliente). Define DataSource, dialect do SQLite, e entidades JPA.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <persistence xmlns="https://jakarta.ee/xml/ns/persistence"
3           xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4           xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence https://jakarta.ee/xml/ns/persistence/persistence_3_0.xsd"
5           version="3.0">
6
7     <!-- BANCO CLIENTE (SQLite local) -->
8     <persistence-unit name="cliente-pu" transaction-type="RESOURCE_LOCAL">
9         <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
10        <class>com.corumbasistemas.corubafood.model.Usuario</class>
11        <class>com.corumbasistemas.corubafood.model.Empresa</class>
12        <properties>
13            <property name="jakarta.persistence.jdbc.driver" value="org.sqlite.JDBC"/>
14            <property name="jakarta.persistence.jdbc.url" value="jdbc:sqlite:cliente.db"/>
15            <property name="hibernate.dialect" value="org.hibernate.community.dialect.SQLiteDialect"/>
16            <property name="hibernate.hbm2ddl.auto" value="update"/>
17        </properties>
18    </persistence-unit>
19
20    <!-- BANCO NUVEM (PostgreSQL VPS) -->
21    <persistence-unit name="nuvem-pu" transaction-type="RESOURCE_LOCAL">
22        <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
23        <class>com.corumbasistemas.corubafood.model.Produto</class>
24        <class>com.corumbasistemas.corubafood.model.Categoria</class>
25        <class>com.corumbasistemas.corubafood.model.Mesa</class>
26        <class>com.corumbasistemas.corubafood.model.Pedido</class>
27        <class>com.corumbasistemas.corubafood.model.ItemPedido</class>
28        <class>com.corumbasistemas.corubafood.model.Pagamento</class>
29        <class>com.corumbasistemas.corubafood.model.PedidoDelivery</class>
30        <class>com.corumbasistemas.corubafood.model.ItemPedidoDelivery</class>
31        <properties>
32            <property name="jakarta.persistence.jdbc.driver" value="org.postgresql.Driver"/>
33            <property name="jakarta.persistence.jdbc.url"
34                value="jdbc:postgresql://${DB_HOST:localhost}:${DB_PORT:5432}/${DB_NAME:corumba}"/>
35            <property name="jakarta.persistence.jdbc.user" value="${DB_USER:corumba}"/>
36            <property name="jakarta.persistence.jdbc.password" value="${DB_PASS:corumba2025}"/>
37            <property name="hibernate.dialect" value="org.hibernate.dialect.PostgreSQLDialect"/>
38            <property name="hibernate.hbm2ddl.auto" value="update"/>
39        </properties>
40    </persistence-unit>
41
42 </persistence>
43
```

### Explicacao:

#### Linha 1-10

XML header, persistence version 2.2, xmlns schema location

#### Linha 12-20

persistence-unit name='clientePU', transaction-type RESOURCE\_LOCAL. Provider: org.hibernate.jpa.HibernatePersistenceProvider

#### Linha 22-35

properties: hibernate.connection.driver\_class (SQLite), hibernate.connection.url (jdbc:sqlite:coruba-food.db), hibernate.dialect (SQLiteDialect), hibernate.hbm2ddl.auto update

#### Linha 36-43

class: lista de entidades JPA (Usuario, Produto, Pedido, ItemPedido, Mesa, Categoria, Pagamento, PedidoDelivery, ItemPedidoDelivery, Empresa)

## 4c. logback.xml

logback.xml

logback.xml • 27 linhas

 Configuracao de logging. Define appenders (console, file), niveis de log por pacote, e formato das mensagens.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <configuration>
3   <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
4     <encoder>
5       <pattern>%d{HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n</pattern>
6     </encoder>
7   </appender>
8
9   <!-- Desativar logs verbose do Hibernate -->
10  <logger name="org.hibernate" level="WARN"/>
11  <logger name="org.hibernate.orm.results" level="OFF"/>
12  <logger name="org.hibernate.sql.results" level="OFF"/>
13  <logger name="org.hibernate.resource.jdbc" level="OFF"/>
14  <logger name="org.hibernate.engine.jdbc" level="OFF"/>
15  <logger name="org.hibernate.cache" level="OFF"/>
16  <logger name="org.hibernate.stat" level="OFF"/>
17  <logger name="org.hibernate.hql" level="OFF"/>
18  <logger name="org.hibernate.prepared" level="OFF"/>
19
20  <!-- Manter INFO da aplicacao -->
21  <logger name="com.corumbasistemas" level="INFO"/>
22
23  <root level="INFO">
24    <appender-ref ref="CONSOLE"/>
25  </root>
26 </configuration>
27
```



## 5. Modelos de Dado (Entities JPA)

### 5.a. model/Usuario.java

#### Usuario.java

coruba-food/src/main/java/com/corumbasistemas/corubafood/model/Usuario.java • 91 linhas

 Entidade Usuario. Representa usuarios/funcionarios do sistema. Campos: id, nome, login, senha (hash), perfil (cargo), ativo.

```
1 package com.corumbasistemas.corubafood.model;
2
3 import jakarta.persistence.*;
4 import java.time.LocalDateTime;
5
6 /**
7  * Usuario - BANCO CLIENTE (SQLite)
8  * Guarda credenciais de acesso
9  */
10 @Entity
11 @Table(name = "usuario_cliente")
12 public class Usuario {
13
14     public enum Papel { ADMIN, GERENTE, GARCOM, CAIXA, COZINHA }
15
16     @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
17     private Long id;
18
19     @Column(nullable = false, length = 100)
20     private String nome;
21
22     @Column(nullable = false, length = 50, unique = true)
23     private String username;
24
25     @Column(nullable = false, length = 200)
26     private String senha; // BCrypt hash
27
28     @Enumerated(EnumType.STRING)
29     @Column(nullable = false, length = 20)
30     private Papel papel;
31
32     // CNPJ da empresa (referencia ao banco cliente)
33     @Column(nullable = false, length = 18)
34     private String empresaDocumento;
35
36     @Column(nullable = false)
37     private Boolean ativo = true;
38
39     @Column(name = "created_at")
40     private LocalDateTime createdAt;
41
42     // Ambiente: cloud ou local
43     @Column(length = 10)
44     private String ambiente = "local";
45
46     @PrePersist
47     protected void onCreate() {
48         createdAt = LocalDateTime.now();
49         if (ambiente == null) ambiente = "local";
50     }
51
52     // Construtor padrão
53     public Usuario() {}
54
55     // Construtor
56     public Usuario(String nome, String username, String senha, Papel papel, String empresaDocumento) {
57         this.nome = nome;
58         this.username = username;
59         this.senha = senha;
60         this.papel = papel;
61         this.empresaDocumento = empresaDocumento;
62         this.ativo = true;
63         this.ambiente = "local";
64     }
65
66     // Getters/Setters
67     public Long getId() { return id; }
68     public void setId(Long id) { this.id = id; }
69     public String getNome() { return nome; }
70     public void setNome(String nome) { this.nome = nome; }
71     public String getUsername() { return username; }
72     public void setUsername(String username) { this.username = username; }
73     public String getSenha() { return senha; }
74     public void setSenha(String senha) { this.senha = senha; }
75     public Papel getPapel() { return papel; }
76     public void setPapel(Papel papel) { this.papel = papel; }
77     public String getEmpresaDocumento() { return empresaDocumento; }
78     public void setEmpresaDocumento(String empresaDocumento) { this.empresaDocumento = empresaDocumento; }
79     public Boolean getAtivo() { return ativo; }
80     public void setAtivo(Boolean ativo) { this.ativo = ativo; }
81     public LocalDateTime getCreatedAt() { return createdAt; }
82     public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }
83 }
```

```

84     public String getAmbiente() { return ambiente; }
85     public void setAmbiente(String ambiente) { this.ambiente = ambiente; }
86
87     @Override
88     public String toString() {
89         return "%s (%s) - %s".formatted(nome, username, papel);
90     }
91 }

```

## 5.b. model/Produto.java



Produto.java

coruba-food/src/main/java/com/corumbasistemas/corubafood/model/Produto.java • 84 linhas



Entidade Produto. Produtos do cardápio. Campos: id, nome, descricao, preco, categoria, ativo, estoque.

```

1  package com.corumbasistemas.corubafood.model;
2
3  import jakarta.persistence.*;
4  import java.math.BigDecimal;
5  import java.time.LocalDateTime;
6
7  /**
8   * Produto - BANCO NUVEM (PostgreSQL)
9   * Cardápio completo do restaurante
10  */
11  @Entity
12  @Table(name = "produto_nuvem", uniqueConstraints = {
13      @UniqueConstraint(columnNames = {"codigo", "empresa_documento"}, name = "uk_produto_cod_emp")
14  })
15  public class Produto {
16
17      @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
18      private Long id;
19
20      @Column(nullable = false, length = 20)
21      private String codigo;
22
23      @Column(nullable = false, length = 200)
24      private String nome;
25
26      @Column(length = 500)
27      private String descricao;
28
29      @Column(nullable = false, precision = 10, scale = 2)
30      private BigDecimal preco;
31
32      @Column(nullable = false, length = 18)
33      private String empresaDocumento; // CNPJ da empresa dona
34
35      @Column(length = 18)
36      private String categoriaDocumento;
37
38      @Column(nullable = false)
39      private Boolean ativo = true;
40
41      @Column(name = "created_at")
42      private LocalDateTime createdAt;
43
44      @PrePersist
45      protected void onCreate() {
46          createdAt = LocalDateTime.now();
47      }
48
49      public Produto() {}
50
51      public Produto(String codigo, String nome, BigDecimal preco, String empresaDocumento) {
52          this.codigo = codigo;
53          this.nome = nome;
54          this.preco = preco;
55          this.empresaDocumento = empresaDocumento;
56          this.ativo = true;
57      }
58
59      // Getters/Setters
60      public Long getId() { return id; }
61      public void setId(Long id) { this.id = id; }
62      public String getCodigo() { return codigo; }
63      public void setCodigo(String codigo) { this.codigo = codigo; }
64      public String getNome() { return nome; }
65      public void setNome(String nome) { this.nome = nome; }
66      public String getDescricao() { return descricao; }
67      public void setDescricao(String descricao) { this.descricao = descricao; }
68      public BigDecimal getPreco() { return preco; }
69      public void setPreco(BigDecimal preco) { this.preco = preco; }
70      public String getEmpresaDocumento() { return empresaDocumento; }
71      public void setEmpresaDocumento(String empresaDocumento) { this.empresaDocumento = empresaDocumento; }
72      public String getCategoriaDocumento() { return categoriaDocumento; }
73      public void setCategoriaDocumento(String categoriaDocumento) { this.categoriaDocumento = categoriaDocumento; }
74      public Boolean getAtivo() { return ativo; }
75      public void setAtivo(Boolean ativo) { this.ativo = ativo; }
76      public LocalDateTime getCreatedAt() { return createdAt; }
77      public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }
78

```

```

79     @Override
80     public String toString() {
81         return "%s - R$ %s".formatted(nome, preco);
82     }
83 }
84

```

### 5.c. model/Categoria.java

#### Categoria.java

coruba-food/src/main/java/com/corumbasistemas/corubafood/model/Categoria.java • 27 linhas

 Entidade Categoria. Categorias de produtos (Bebidas, Pratos, Sobremesas, etc).

```

1  package com.corumbasistemas.corubafood.model;
2
3  import jakarta.persistence.*;
4  import java.time.LocalDateTime;
5
6  @Entity
7  @Table(name = "categoria", uniqueConstraints = {
8      @UniqueConstraint(columnNames = {"nome", "empresa_id"}, name = "uk_categoria_nome_empresa")
9  })
10 public class Categoria {
11     @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
12     @Column(nullable = false, length = 100) private String nome;
13     @Column(length = 255) private String descricao;
14     @Column(name = "ordem") private Integer ordem = 0;
15     @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "empresa_id", nullable = false) private Empresa empresa;
16     @Column(name = "ativo", nullable = false) private Boolean ativo = true;
17     @Column(name = "created_at") private LocalDateTime createdAt;
18     @PrePersist protected void onCreate() { createdAt = LocalDateTime.now(); }
19     public Long getId() { return id; } public void setId(Long id) { this.id = id; }
20     public String getNome() { return nome; } public void setNome(String n) { this.nome = n; }
21     public String getDescricao() { return descricao; } public void setDescricao(String d) { this.descricao = d; }
22     public Integer getOrdem() { return ordem; } public void setOrdem(Integer o) { this.ordem = o; }
23     public Empresa getEmpresa() { return empresa; } public void setEmpresa(Empresa e) { this.empresa = e; }
24     public Boolean getAtivo() { return ativo; } public void setAtivo(Boolean a) { this.ativo = a; }
25     public LocalDateTime getCreatedAt() { return createdAt; } public void setCreatedAt(LocalDateTime c) { this.createdAt = c; }
26 }
27

```

### 5.d. model/Mesa.java

#### Mesa.java

coruba-food/src/main/java/com/corumbasistemas/corubafood/model/Mesa.java • 79 linhas

 Entidade Mesa. Mesas do restaurante. Campos: id, numero, status (LIVRE/OCUPADA/RESERVADA), capacidade.

```

1  package com.corumbasistemas.corubafood.model;
2
3  import jakarta.persistence.*;
4  import java.time.LocalDateTime;
5
6  /**
7   * Mesa - BANCO NUVEM (PostgreSQL)
8   * Mesas do restaurante, identificadas pelo CNPJ da empresa
9   */
10 @Entity
11 @Table(name = "mesa_nuvem", uniqueConstraints = {
12     @UniqueConstraint(columnNames = {"numero", "empresa_documento"}, name = "uk_mesa_num_emp")
13 })
14 public class Mesa {
15
16     public enum Status { LIVRE, OCUPADA, RESERVADA }
17
18     public String getDescricao() {
19         return switch (this.status) {
20             case LIVRE -> "Disponível";
21             case OCUPADA -> "Em uso";
22             case RESERVADA -> "Reservada";
23         };
24     }
25
26     @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
27     private Long id;
28
29     @Column(nullable = false)
30     private Integer numero;
31
32     @Column(nullable = false)
33     private Integer capacidade;
34
35     @Enumerated(EnumType.STRING)
36     @Column(nullable = false, length = 20)
37     private Status status = Status.LIVRE;
38
39     // CNPJ da empresa (banco nuvem não tem FK para banco cliente)
40     @Column(nullable = false, length = 18, name = "empresa_documento")
41     private String empresaDocumento;
42
43     @Column(name = "created_at")
44     private LocalDateTime createdAt;
45

```

```

46     @PrePersist
47     protected void onCreate() {
48         createdAt = LocalDateTime.now();
49     }
50
51     public Mesa() {}
52
53     public Mesa(Integer numero, Integer capacidade, String empresaDocumento) {
54         this.numero = numero;
55         this.capacidade = capacidade;
56         this.empresaDocumento = empresaDocumento;
57         this.status = Status.LIVRE;
58     }
59
60     // Getters/Setters
61     public Long getId() { return id; }
62     public void setId(Long id) { this.id = id; }
63     public Integer getNumero() { return numero; }
64     public void setNumero(Integer numero) { this.numero = numero; }
65     public Integer getCapacidade() { return capacidade; }
66     public void setCapacidade(Integer capacidade) { this.capacidade = capacidade; }
67     public Status getStatus() { return status; }
68     public void setStatus(Status status) { this.status = status; }
69     public String getEmpresaDocumento() { return empresaDocumento; }
70     public void setEmpresaDocumento(String empresaDocumento) { this.empresaDocumento = empresaDocumento; }
71     public LocalDateTime getCreatedAt() { return createdAt; }
72     public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }
73
74     @Override
75     public String toString() {
76         return "Mesa %d (%d lugares) - %s".formatted(numero, capacidade, status);
77     }
78 }
79

```

## 5.e. model/Pedido.java

### Pedido.java

coruba-food/src/main/java/com/corubasistemas/corubafood/model/Pedido.java • 76 linhas

 Entidade Pedido. Pedidos realizados. Campos: id, mesa, itens, status, dataHora, total, usuario.

```

1  package com.corubasistemas.corubafood.model;
2
3  import jakarta.persistence.*;
4  import java.math.BigDecimal;
5  import java.time.LocalDateTime;
6  import java.util.ArrayList;
7  import java.util.List;
8
9  /**
10   * Pedido - BANCO NUVEM (PostgreSQL)
11   * Pedidos do restaurante, identificados pelo CNPJ da empresa
12   */
13  @Entity
14  @Table(name = "pedido_nuvem")
15  public class Pedido {
16
17      public enum Status { ABERTO, EM_PREPARO, PRONTO, ENTREGUE, CANCELADO }
18
19      @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
20      private Long id;
21
22      // IDs de referencia (sem FK para outros bancos)
23      @Column(name = "mesa_id")
24      private Long mesaId;
25
26      @Column(name = "usuario_id")
27      private Long usuarioId;
28
29      // CNPJ da empresa (chave de sincronizacao)
30      @Column(nullable = false, length = 18, name = "empresa_documento")
31      private String empresaDocumento;
32
33      @Enumerated(EnumType.STRING)
34      @Column(nullable = false, length = 20)
35      private Status status = Status.ABERTO;
36
37      @Column(precision = 10, scale = 2)
38      private BigDecimal total = BigDecimal.ZERO;
39
40      @Column(name = "created_at")
41      private LocalDateTime createdAt;
42
43      @OneToMany(mappedBy = "pedido", cascade = CascadeType.ALL)
44      private List<ItemPedido> itens = new ArrayList<>();
45
46      @PrePersist
47      protected void onCreate() {
48          createdAt = LocalDateTime.now();
49      }
50
51      public Pedido() {}
52
53      // Getters/Setters

```

```

54 public Long getId() { return id; }
55 public void setId(Long id) { this.id = id; }
56 public Long getMesaId() { return mesaId; }
57 public void setMesaId(Long mesaId) { this.mesaId = mesaId; }
58 public Long getUsuarioId() { return usuarioId; }
59 public void setUsuarioId(Long usuarioId) { this.usuarioId = usuarioId; }
60 public String getEmpresaDocumento() { return empresaDocumento; }
61 public void setEmpresaDocumento(String empresaDocumento) { this.empresaDocumento = empresaDocumento; }
62 public Status getStatus() { return status; }
63 public void setStatus(Status status) { this.status = status; }
64 public BigDecimal getTotal() { return total; }
65 public void setTotal(BigDecimal total) { this.total = total; }
66 public LocalDateTime getCreatedAt() { return createdAt; }
67 public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }
68 public List<ItemPedido> getItens() { return itens; }
69 public void setItens(List<ItemPedido> itens) { this.itens = itens; }
70
71 @Override
72 public String toString() {
73     return "Pedido #%-d - %-s - R$ %s".formatted(id, status, total);
74 }
75 }
76

```

## 5.f. model/ItemPedido.java

### ItemPedido.java

coruba-food/src/main/java/com/corumbasistemas/corubafood/model/ItemPedido.java • 54 linhas

 Entidade ItemPedido. Itens de um pedido. Campos: id, pedido, produto, quantidade, precoUnitario, observacao.

```

1 package com.corumbasistemas.corubafood.model;
2
3 import jakarta.persistence.*;
4 import java.math.BigDecimal;
5
6 /**
7  * ItemPedido - BANCO NUVEM (PostgreSQL)
8  */
9 @Entity
10 @Table(name = "item_pedido_nuem")
11 public class ItemPedido {
12
13     @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
14     private Long id;
15
16     @ManyToOne(fetch = FetchType.LAZY)
17     @JoinColumn(name = "pedido_id", nullable = false)
18     private Pedido pedido;
19
20     @ManyToOne(fetch = FetchType.LAZY)
21     @JoinColumn(name = "produto_id", nullable = false)
22     private Produto produto;
23
24     @Column(nullable = false)
25     private Integer quantidade;
26
27     @Column(nullable = false, precision = 10, scale = 2)
28     private BigDecimal precoUnitario;
29
30     @Column(nullable = false, precision = 10, scale = 2)
31     private BigDecimal subtotal;
32
33     @Column(length = 18, name = "empresa_documento")
34     private String empresaDocumento;
35
36     public ItemPedido() {}
37
38     // Getters/Setters
39     public Long getId() { return id; }
40     public void setId(Long id) { this.id = id; }
41     public Pedido getPedido() { return pedido; }
42     public void setPedido(Pedido pedido) { this.pedido = pedido; }
43     public Produto getProduto() { return produto; }
44     public void setProduto(Produto produto) { this.produto = produto; }
45     public Integer getQuantidade() { return quantidade; }
46     public void setQuantidade(Integer quantidade) { this.quantidade = quantidade; }
47     public BigDecimal getPrecoUnitario() { return precoUnitario; }
48     public void setPrecoUnitario(BigDecimal precoUnitario) { this.precoUnitario = precoUnitario; }
49     public BigDecimal getSubtotal() { return subtotal; }
50     public void setSubtotal(BigDecimal subtotal) { this.subtotal = subtotal; }
51     public String getEmpresaDocumento() { return empresaDocumento; }
52     public void setEmpresaDocumento(String empresaDocumento) { this.empresaDocumento = empresaDocumento; }
53 }
54

```

## 5.g. model/PedidoDelivery.java

### PedidoDelivery.java

coruba-food/src/main/java/com/corumbasistemas/corubafood/model/PedidoDelivery.java • 167 linhas

 Entidade PedidoDelivery. Pedidos de entrega. Campos: id, clienteNome, clienteTelefone, endereco, itens, status, total, taxaEntrega.

```

1 package com.corumbasistemas.corubafood.model;

```

```

2
3 import jakarta.persistence.*;
4 import java.math.BigDecimal;
5 import java.time.LocalDateTime;
6 import java.util.ArrayList;
7 import java.util.List;
8
9 /**
10  * PedidoDelivery - Pedidos vindos de plataformas de delivery (iFood, 99Food).
11  */
12 @Entity
13 @Table(name = "pedidos_delivery")
14 public class PedidoDelivery {
15
16     public enum Plataforma { IF00D, F00D99 }
17     public enum StatusPedido { NOVO, CONFIRMADO, PREPARANDO, PRONTO, SAIU_ENTREGA, ENTREGUE, CANCELADO }
18
19     @Id
20     @GeneratedValue(strategy = GenerationType.IDENTITY)
21     private Long id;
22
23     @Column(name = "codigo_plataforma", nullable = false, length = 50)
24     private String codigoPlataforma;
25
26     @Enumerated(EnumType.STRING)
27     @Column(nullable = false, length = 20)
28     private Plataforma plataforma;
29
30     @Column(name = "nome_cliente", nullable = false, length = 200)
31     private String nomeCliente;
32
33     @Column(name = "telefone_cliente", length = 20)
34     private String telefoneCliente;
35
36     @Column(name = "endereco_entrega", length = 500)
37     private String enderecoEntrega;
38
39     @OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)
40     @JoinColumn(name = "pedido_delivery_id")
41     private List<ItemPedidoDelivery> itens = new ArrayList<>();
42
43     @Column(name = "valor_itens", nullable = false, precision = 10, scale = 2)
44     private BigDecimal valorItens;
45
46     @Column(name = "taxa_entrega", precision = 10, scale = 2)
47     private BigDecimal taxaEntrega = BigDecimal.ZERO;
48
49     @Column(name = "valor_total", nullable = false, precision = 10, scale = 2)
50     private BigDecimal valorTotal;
51
52     @Enumerated(EnumType.STRING)
53     @Column(nullable = false, length = 20)
54     private StatusPedido status = StatusPedido.NOVO;
55
56     @Column(name = "forma_pagamento", length = 50)
57     private String formaPagamento;
58
59     @Column(name = "observacoes", length = 500)
60     private String observacoes;
61
62     @ManyToOne(fetch = FetchType.LAZY)
63     @JoinColumn(name = "empresa_id", nullable = false)
64     private Empresa empresa;
65
66     @Column(name = "data_criacao", nullable = false)
67     private LocalDateTime dataCriacao = LocalDateTime.now();
68
69     @Column(name = "data_confirmacao")
70     private LocalDateTime dataConfirmacao;
71
72     @Column(name = "data_pronto")
73     private LocalDateTime dataPronto;
74
75     @Column(name = "data_entrega")
76     private LocalDateTime dataEntrega;
77
78     @Column(name = "data_cancelamento")
79     private LocalDateTime dataCancelamento;
80
81     @Column(name = "motivo_cancelamento", length = 500)
82     private String motivoCancelamento;
83
84     public PedidoDelivery() {}
85
86     public PedidoDelivery(Plataforma plataforma, String codigoPlataforma, String nomeCliente, Empresa empresa) {
87         this.plataforma = plataforma;
88         this.codigoPlataforma = codigoPlataforma;
89         this.nomeCliente = nomeCliente;
90         this.empresa = empresa;
91         this.valorItens = BigDecimal.ZERO;
92         this.valorTotal = BigDecimal.ZERO;
93         this.status = StatusPedido.NOVO;
94     }
95
96     public void calcularTotal() {
97         if (itens == null || itens.isEmpty()) {

```

```

98         this.valorItens = BigDecimal.ZERO;
99     } else {
100         this.valorItens = itens.stream()
101             .map(ItemPedidoDelivery::getSubtotal)
102             .reduce(BigDecimal.ZERO, BigDecimal::add);
103     }
104     BigDecimal taxa = this.taxaEntrega != null ? this.taxaEntrega : BigDecimal.ZERO;
105     this.valorTotal = this.valorItens.add(taxa);
106 }
107
108 public boolean avancarStatus() {
109     switch (this.status) {
110         case NOV0: this.status = StatusPedido.CONFIRMADO; this.dataConfirmacao = LocalDateTime.now(); return true;
111         case CONFIRMADO: this.status = StatusPedido.PREPARANDO; return true;
112         case PREPARANDO: this.status = StatusPedido.PRONTO; this.dataPronto = LocalDateTime.now(); return true;
113         case PRONTO: this.status = StatusPedido.SAIU_ENTREGA; return true;
114         case SAIU_ENTREGA: this.status = StatusPedido.ENTREGUE; this.dataEntrega = LocalDateTime.now(); return true;
115         default: return false;
116     }
117 }
118
119 public void cancelar(String motivo) {
120     this.status = StatusPedido.CANCELADO;
121     this.motivoCancelamento = motivo;
122     this.dataCancelamento = LocalDateTime.now();
123 }
124
125 // Getters e Setters
126 public Long getId() { return id; }
127 public void setId(Long id) { this.id = id; }
128 public String getCodigoPlataforma() { return codigoPlataforma; }
129 public void setCodigoPlataforma(String c) { this.codigoPlataforma = c; }
130 public Plataforma getPlataforma() { return plataforma; }
131 public void setPlataforma(Plataforma p) { this.plataforma = p; }
132 public String getNomeCliente() { return nomeCliente; }
133 public void setNomeCliente(String n) { this.nomeCliente = n; }
134 public String getTelefoneCliente() { return telefoneCliente; }
135 public void setTelefoneCliente(String t) { this.telefoneCliente = t; }
136 public String getEnderecoEntrega() { return enderecoEntrega; }
137 public void setEnderecoEntrega(String e) { this.enderecoEntrega = e; }
138 public List<ItemPedidoDelivery> getItens() { return itens; }
139 public void setItens(List<ItemPedidoDelivery> i) { this.itens = i; }
140 public BigDecimal getValorItens() { return valorItens; }
141 public void setValorItens(BigDecimal v) { this.valorItens = v; }
142 public BigDecimal getTaxaEntrega() { return taxaEntrega; }
143 public void setTaxaEntrega(BigDecimal t) { this.taxaEntrega = t; }
144 public BigDecimal getValorTotal() { return valorTotal; }
145 public void setValorTotal(BigDecimal v) { this.valorTotal = v; }
146 public StatusPedido getStatus() { return status; }
147 public void setStatus(StatusPedido s) { this.status = s; }
148 public String getFormaPagamento() { return formaPagamento; }
149 public void setFormaPagamento(String f) { this.formaPagamento = f; }
150 public String getObservacoes() { return observacoes; }
151 public void setObservacoes(String o) { this.observacoes = o; }
152 public Empresa getEmpresa() { return empresa; }
153 public void setEmpresa(Empresa e) { this.empresa = e; }
154 public LocalDateTime getDataCriacao() { return dataCriacao; }
155 public void setDataCriacao(LocalDateTime d) { this.dataCriacao = d; }
156 public LocalDateTime getDataConfirmacao() { return dataConfirmacao; }
157 public void setDataConfirmacao(LocalDateTime d) { this.dataConfirmacao = d; }
158 public LocalDateTime getDataPronto() { return dataPronto; }
159 public void setDataPronto(LocalDateTime d) { this.dataPronto = d; }
160 public LocalDateTime getDataEntrega() { return dataEntrega; }
161 public void setDataEntrega(LocalDateTime d) { this.dataEntrega = d; }
162 public LocalDateTime getDataCancelamento() { return dataCancelamento; }
163 public void setDataCancelamento(LocalDateTime d) { this.dataCancelamento = d; }
164 public String getMotivoCancelamento() { return motivoCancelamento; }
165 public void setMotivoCancelamento(String m) { this.motivoCancelamento = m; }
166 }
167

```

## 5.h. model/ItemPedidoDelivery.java



ItemPedidoDelivery.java

coruba-food/src/main/java/com/corumbasistemas/corubafood/model/ItemPedidoDelivery.java • 98  
linhas



Entidade ItemPedidoDelivery. Itens do pedido de delivery.

```

1 package com.corumbasistemas.corubafood.model;
2
3 import jakarta.persistence.*;
4 import java.math.BigDecimal;
5
6 /**
7  * ItemPedidoDelivery - Item individual de um pedido de delivery.
8  *
9  * Cada item representa um produto do cardápio que foi
10  * pedido através do iFood ou 99Food.
11  */
12 @Entity
13 @Table(name = "itens_pedido_delivery")
14 public class ItemPedidoDelivery {
15
16

```

```

17  @Id
18  @GeneratedValue(strategy = GenerationType.IDENTITY)
19  private Long id;
20
21  // Nome do produto no momento do pedido (pode mudar no cardápio depois)
22  @Column(name = "nome_produto", nullable = false, length = 200)
23  private String nomeProduto;
24
25  // Quantidade pedida
26  @Column(nullable = false)
27  private Integer quantidade;
28
29  // Preço unitário no momento do pedido
30  @Column(name = "preco_unitario", nullable = false, precision = 10, scale = 2)
31  private BigDecimal precoUnitario;
32
33  // Subtotal = quantidade x preço unitário
34  @Column(nullable = false, precision = 10, scale = 2)
35  private BigDecimal subtotal;
36
37  // Observações específicas do item (ex: "sem cebola", "mal passado")
38  @Column(name = "observacoes", length = 300)
39  private String observacoes;
40
41  // Combo/adicionais
42  @Column(name = "adicionais", length = 500)
43  private String adicionais;
44
45  // Construtor padrão
46  public ItemPedidoDelivery() {}
47
48  // Construtor com campos obrigatórios
49  public ItemPedidoDelivery(String nomeProduto, Integer quantidade, BigDecimal precoUnitario) {
50      this.nomeProduto = nomeProduto;
51      this.quantidade = quantidade;
52      this.precoUnitario = precoUnitario;
53      calcularSubtotal();
54  }
55
56  /**
57   * Calcula o subtotal do item
58   */
59  public void calcularSubtotal() {
60      if (this.precoUnitario != null && this.quantidade != null) {
61          this.subtotal = this.precoUnitario.multiply(BigDecimal.valueOf(this.quantidade));
62      }
63  }
64
65  // --- Getters e Setters ---
66
67  public Long getId() { return id; }
68  public void setId(Long id) { this.id = id; }
69
70  public String getNomeProduto() { return nomeProduto; }
71  public void setNomeProduto(String nomeProduto) { this.nomeProduto = nomeProduto; }
72
73  public Integer getQuantidade() { return quantidade; }
74  public void setQuantidade(Integer quantidade) {
75      this.quantidade = quantidade;
76      calcularSubtotal();
77  }
78
79  public BigDecimal getPrecoUnitario() { return precoUnitario; }
80  public void setPrecoUnitario(BigDecimal precoUnitario) {
81      this.precoUnitario = precoUnitario;
82      calcularSubtotal();
83  }
84
85  public BigDecimal getSubtotal() { return subtotal; }
86  public void setSubtotal(BigDecimal subtotal) { this.subtotal = subtotal; }
87
88  public String getObservacoes() { return observacoes; }
89  public void setObservacoes(String observacoes) { this.observacoes = observacoes; }
90
91  public String getAdicionais() { return adicionais; }
92  public void setAdicionais(String adicionais) { this.adicionais = adicionais; }
93
94  @Override
95  public String toString() {
96      return quantidade + "x " + nomeProduto + " = R$ " + subtotal;
97  }
98  }

```

## 5.i. model/Pagamento.java



### Pagamento.java

coruba-food/src/main/java/com/corumbasistemas/corubafood/model/Pagamento.java • 29 linhas



*Entidade Pagamento. Registros de pagamento. Campos: id, pedido, formaPagamento, valor, dataHora, status.*

```

1  package com.corumbasistemas.corubafood.model;
2
3  import jakarta.persistence.*;
4  import java.math.BigDecimal;

```



```

5 import java.time.LocalDateTime;
6
7 @Entity
8 @Table(name = "pagamento")
9 public class Pagamento {
10     public enum FormaPagamento { DINHEIRO, CARTAO_CREDITO, CARTAO_DEBITO, PIX, VALE_REFEICAO }
11     @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
12     @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "pedido_id", nullable = false) private Pedido pedido;
13     @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "empresa_id", nullable = false) private Empresa empresa;
14     @Enumerated(EnumType.STRING) @Column(nullable = false, length = 30) private FormaPagamento formaPagamento;
15     @Column(nullable = false, precision = 10, scale = 2) private BigDecimal valor;
16     @Column(precision = 10, scale = 2) private BigDecimal desconto = BigDecimal.ZERO;
17     @Column(name = "desconto_autorizado_por", length = 255) private String descontoAutorizadoPor;
18     @Column(name = "created_at") private LocalDateTime createdAt;
19     @PrePersist protected void onCreate() { createdAt = LocalDateTime.now(); }
20     public Long getId() { return id; } public void setId(Long id) { this.id = id; }
21     public Pedido getPedido() { return pedido; } public void setPedido(Pedido p) { this.pedido = p; }
22     public Empresa getEmpresa() { return empresa; } public void setEmpresa(Empresa e) { this.empresa = e; }
23     public FormaPagamento getFormaPagamento() { return formaPagamento; } public void setFormaPagamento(FormaPagamento f)
24     { this.formaPagamento = f; }
25     public BigDecimal getValor() { return valor; } public void setValor(BigDecimal v) { this.valor = v; }
26     public BigDecimal getDesconto() { return desconto; } public void setDesconto(BigDecimal d) { this.desconto = d; }
27     public String getDescontoAutorizadoPor() { return descontoAutorizadoPor; } public void setDescontoAutorizadoPor(String d)
28     { this.descontoAutorizadoPor = d; }
29     public LocalDateTime getCreatedAt() { return createdAt; } public void setCreatedAt(LocalDateTime c) { this.createdAt = c; }
30 }

```

## 5.j. model/Empresa.java

 **Empresa.java** coruba-food/src/main/java/com/corumbasistemas/corubafood/model/Empresa.java • 102 linhas

 **Entidade Empresa.** Dados da empresa cliente. Campos: id, razaoSocial, cnpj, email, telefone.

```

1 package com.corumbasistemas.corubafood.model;
2
3 import jakarta.persistence.*;
4 import java.time.LocalDateTime;
5
6 /**
7  * Empresa - BANCO CLIENTE (SQLite)
8  * Guarda dados de autenticação e identificação
9  */
10 @Entity
11 @Table(name = "empresa_cliente", uniqueConstraints = {
12     @UniqueConstraint(columnNames = "documento", name = "uk_empresa_doc")
13 })
14 public class Empresa {
15
16     public enum Tipo { RESTAURANTE, PIZZARIA, HAMBURGUERIA, CAFETERIA, BAR, OUTRO }
17
18     @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
19     private Long id;
20
21     @Column(nullable = false, length = 200)
22     private String nome;
23
24     @Column(nullable = false, length = 18, unique = true)
25     private String documento; // CNPJ ou CPF
26
27     @Enumerated(EnumType.STRING)
28     @Column(length = 20)
29     private Tipo tipo = Tipo.RESTAURANTE;
30
31     @Column(length = 20)
32     private String telefone;
33
34     @Column(length = 300)
35     private String endereco;
36
37     @Column(length = 100)
38     private String cidade;
39
40     @Column(length = 2)
41     private String estado;
42
43     @Column(nullable = false)
44     private Boolean ativo = true;
45
46     // Credenciais da API da nuvem
47     @Column(length = 500)
48     private String apiUrl;
49
50     @Column(length = 200)
51     private String apiKey;
52
53     @Column(name = "created_at")
54     private LocalDateTime createdAt;
55
56     @PrePersist
57     protected void onCreate() {
58         createdAt = LocalDateTime.now();
59     }
60 }

```

```

61 // Construtor padrão
62 public Empresa() {}
63
64 // Construtor
65 public Empresa(String nome, String documento) {
66     this.nome = nome;
67     this.documento = documento;
68     this.ativo = true;
69 }
70
71 // Getters/Setters
72 public Long getId() { return id; }
73 public void setId(Long id) { this.id = id; }
74 public String getNome() { return nome; }
75 public void setNome(String nome) { this.nome = nome; }
76 public String getDocumento() { return documento; }
77 public void setDocumento(String documento) { this.documento = documento; }
78 public Tipo getTipo() { return tipo; }
79 public void setTipo(Tipo tipo) { this.tipo = tipo; }
80 public String getTelefone() { return telefone; }
81 public void setTelefone(String telefone) { this.telefone = telefone; }
82 public String getEndereco() { return endereco; }
83 public void setEndereco(String endereco) { this.endereco = endereco; }
84 public String getCidade() { return cidade; }
85 public void setCidade(String cidade) { this.cidade = cidade; }
86 public String getEstado() { return estado; }
87 public void setEstado(String estado) { this.estado = estado; }
88 public Boolean getAtivo() { return ativo; }
89 public void setAtivo(Boolean ativo) { this.ativo = ativo; }
90 public String getApiUrl() { return apiUrl; }
91 public void setApiUrl(String apiUrl) { this.apiUrl = apiUrl; }
92 public String getApiKey() { return apiKey; }
93 public void setApiKey(String apiKey) { this.apiKey = apiKey; }
94 public LocalDateTime getCreatedAt() { return createdAt; }
95 public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }
96
97 @Override
98 public String toString() {
99     return "%s (%s)".formatted(nome, documento);
100 }
101 }
102

```

#### Explicacao:

##### **@Entity, @Table**

Mapeia classe Java para tabela no banco. @Table permite customizar o nome da tabela

##### **@Id, @GeneratedValue**

Define chave primaria e estrategia de geracao (AUTO = auto-increment)

##### **@ManyToOne, @OneToMany**

Define relacionamentos. @ManyToOne = muitos para um. @OneToMany = um para muitos

##### **@JoinColumn**

Define a coluna de chave estrangeira no banco

## 6. Repositorios (Data Access Layer)

### 6.a. repository/ProdutoRepository.java



ProdutoRepository.java

coruba-food/src/main/java/com/corumbasistemas/corubafood/repository/ProdutoRepository.java • 82  
linhas



Repositorio de produtos. Metodos: findAll, findById, findByCategoria, save, delete, buscarPorNome.

```
1 package com.corumbasistemas.corubafood.repository;
2
3 import com.corumbasistemas.corubafood.model.*;
4 import com.corumbasistemas.corubafood.util.JPAUtil;
5 import jakarta.persistence.*;
6 import java.util.List;
7
8 /**
9  * ProdutoRepository - Banco NUVEM (PostgreSQL)
10  * Usa empresaDocumento (CNPJ) em vez de FK
11  */
12 public class ProdutoRepository {
13
14     /**
15      * Buscar todos os produtos de uma empresa (pelo CNPJ)
16      */
17     public List<Produto> buscarTodos(String empresaDocumento) {
18         EntityManager em = JPAUtil.getEMNuvem();
19         try {
20             return em.createQuery(
21                 "SELECT p FROM Produto p WHERE p.empresaDocumento = :edoc AND p.ativo = true ORDER BY p.nome",
22                 Produto.class)
23                 .setParameter("edoc", empresaDocumento)
24                 .getResultList();
25         } finally {
26             em.close();
27         }
28     }
29
30     /**
31      * Buscar produtos por categoria
32      */
33     public List<Produto> buscarPorCategoria(String empresaDocumento, Long categoriaId) {
34         EntityManager em = JPAUtil.getEMNuvem();
35         try {
36             return em.createQuery(
37                 "SELECT p FROM Produto p WHERE p.empresaDocumento = :edoc AND p.categoria.id = :cid AND p.ativo = true ORDER BY
38 p.nome",
39                 Produto.class)
40                 .setParameter("edoc", empresaDocumento)
41                 .setParameter("cid", categoriaId)
42                 .getResultList();
43         } finally {
44             em.close();
45         }
46     }
47
48     /**
49      * Buscar categorias de uma empresa
50      */
51     public List<Categoria> buscarCategorias(String empresaDocumento) {
52         EntityManager em = JPAUtil.getEMNuvem();
53         try {
54             return em.createQuery(
55                 "SELECT c FROM Categoria c WHERE c.empresaDocumento = :edoc ORDER BY c.ordem",
56                 Categoria.class)
57                 .setParameter("edoc", empresaDocumento)
58                 .getResultList();
59         } finally {
60             em.close();
61         }
62     }
63
64     /**
65      * Salvar produto
66      */
67     public Produto salvar(Produto p) {
68         EntityManager em = JPAUtil.getEMNuvem();
69         try {
70             em.getTransaction().begin();
71             if (p.getId() == null) em.persist(p);
72             else p = em.merge(p);
73             em.getTransaction().commit();
74             return p;
75         } catch (Exception e) {
76             if (em.getTransaction().isActive()) em.getTransaction().rollback();
77             throw e;
78         } finally {
79             em.close();
80         }
81     }
82 }
```

```
81 }
82 }
```

## 6.b. repository/PagamentoRepository.java



PagamentoRepository.java

coruba-food/src/main/java/com/corumbasistemas/corubafood/repository/PagamentoRepository.java • 67  
linhas



Repositorio de pagamentos. Metodos: registrar, listarPorPedido, listarPorData, somarTotal.

```
1 package com.corumbasistemas.corubafood.repository;
2
3 import com.corumbasistemas.corubafood.model.Pagamento;
4 import com.corumbasistemas.corubafood.util.JPAUtil;
5 import jakarta.persistence.*;
6 import java.util.List;
7
8 /**
9  * PagamentoRepository - Banco Cliente (SQLite)
10  */
11 public class PagamentoRepository {
12
13     public Pagamento buscarPorId(Long id) {
14         EntityManager em = JPAUtil.getEMCliente();
15         try {
16             return em.find(Pagamento.class, id);
17         } finally {
18             em.close();
19         }
20     }
21
22     public Pagamento buscarPorPedido(Long empresaDoc, Long pid) {
23         EntityManager em = JPAUtil.getEMCliente();
24         try {
25             return em.createQuery(
26                 "SELECT p FROM Pagamento p WHERE p.empresaDocumento = :edoc AND p.pedido.id = :pid",
27                 Pagamento.class)
28                 .setParameter("edoc", empresaDoc)
29                 .setParameter("pid", pid)
30                 .getResultStream()
31                 .findFirst()
32                 .orElse(null);
33         } finally {
34             em.close();
35         }
36     }
37
38     public List<Pagamento> buscarTodos(String empresaDocumento) {
39         EntityManager em = JPAUtil.getEMCliente();
40         try {
41             return em.createQuery(
42                 "SELECT p FROM Pagamento p WHERE p.empresaDocumento = :edoc ORDER BY p.createdAt DESC",
43                 Pagamento.class)
44                 .setParameter("edoc", empresaDocumento)
45                 .getResultList();
46         } finally {
47             em.close();
48         }
49     }
50
51     public Pagamento salvar(Pagamento p) {
52         EntityManager em = JPAUtil.getEMCliente();
53         try {
54             em.getTransaction().begin();
55             if (p.getId() == null) em.persist(p);
56             else p = em.merge(p);
57             em.getTransaction().commit();
58             return p;
59         } catch (Exception e) {
60             if (em.getTransaction().isActive()) em.getTransaction().rollback();
61             throw e;
62         } finally {
63             em.close();
64         }
65     }
66 }
67 }
```

## 6.c. repository/UsuarioRepository.java



UsuarioRepository.java

coruba-food/src/main/java/com/corumbasistemas/corubafood/repository/UsuarioRepository.java • 104  
linhas



Repositorio de usuarios. Metodos: findByLogin, autenticar, listarPorPerfil, salvar.

```
1 package com.corumbasistemas.corubafood.repository;
2
3 import com.corumbasistemas.corubafood.model.Usuario;
4 import com.corumbasistemas.corubafood.util.JPAUtil;
5 import jakarta.persistence.*;
6 import java.util.List;
```

```

7
8 /**
9  * UsuarioRepository - Acesso a dados de usuários (SQLite local)
10 * Usa empresaDocumento (String CNPJ) em vez de FK para Empresa
11 */
12 public class UsuarioRepository {
13
14     /**
15      * Busca usuário por username e CNPJ da empresa
16      */
17     public Usuario buscarPorUsername(String cnpj, String username) {
18         EntityManager em = JPAUtil.getEMCliente();
19         try {
20             return em.createQuery(
21                 "SELECT u FROM Usuario u WHERE u.empresaDocumento=:cnpj AND u.username=:u AND u.ativo=true",
22                 Usuario.class)
23                 .setParameter("cnpj", cnpj)
24                 .setParameter("u", username)
25                 .getSingleResult();
26         } catch (NoResultException e) {
27             return null;
28         } finally {
29             em.close();
30         }
31     }
32
33     /**
34      * Busca usuário por ID
35      */
36     public Usuario buscarPorId(Long id) {
37         EntityManager em = JPAUtil.getEMCliente();
38         try {
39             return em.find(Usuario.class, id);
40         } finally {
41             em.close();
42         }
43     }
44
45     /**
46      * Lista todos os funcionários de uma empresa (por CNPJ)
47      */
48     public List<Usuario> buscarTodosPorCnpj(String cnpj) {
49         EntityManager em = JPAUtil.getEMCliente();
50         try {
51             return em.createQuery(
52                 "SELECT u FROM Usuario u WHERE u.empresaDocumento=:cnpj AND u.ativo=true ORDER BY u.nome",
53                 Usuario.class)
54                 .setParameter("cnpj", cnpj)
55                 .getResultList();
56         } finally {
57             em.close();
58         }
59     }
60
61     /**
62      * Salva ou atualiza usuário
63      */
64     public Usuario salvar(Usuario u) {
65         EntityManager em = JPAUtil.getEMCliente();
66         try {
67             em.getTransaction().begin();
68             if (u.getId() == null) {
69                 em.persist(u);
70             } else {
71                 u = em.merge(u);
72             }
73             em.getTransaction().commit();
74             return u;
75         } catch (Exception e) {
76             if (em.getTransaction().isActive()) em.getTransaction().rollback();
77             throw e;
78         } finally {
79             em.close();
80         }
81     }
82
83     /**
84      * Exclui usuário (soft delete - desativa)
85      */
86     public void desativar(Long id) {
87         EntityManager em = JPAUtil.getEMCliente();
88         try {
89             em.getTransaction().begin();
90             Usuario u = em.find(Usuario.class, id);
91             if (u != null) {
92                 u.setAtivo(false);
93                 em.merge(u);
94             }
95             em.getTransaction().commit();
96         } catch (Exception e) {
97             if (em.getTransaction().isActive()) em.getTransaction().rollback();
98             throw e;
99         } finally {
100             em.close();
101         }
102     }

```



## 7. Servicos (Logica de Negocio)

### 7a. LicencaService.java

 LicencaService.java

service/LicencaService.java • 236 linhas

 *Servico de licenca. Valida licenca online (API VPS) com cache offline de 30 dias. Usa arquivo properties para persistir cache.*

```
1 package com.corumbasistemas.corubafood.service;
2
3 import com.corumbasistemas.corubafood.model.Empresa;
4 import com.corumbasistemas.corubafood.util.JPAUtil;
5 import jakarta.persistence.EntityManager;
6 import org.slf4j.*;
7 import java.io.*;
8 import java.net.*;
9 import java.time.*;
10 import java.time.format.DateTimeFormatter;
11
12 /**
13  * Servico de validacao de licenca do cliente.
14  *
15  * FLUXO:
16  * 1. Cliente ativa com codigo de ativacao (1a vez)
17  * 2. Licenca fica cacheada localmente (30 dias)
18  * 3. A cada login, valida com VPS se online
19  * 4. Se offline, usa cache local (ate 30 dias)
20  * 5. Se expirou, bloqueia o sistema
21  */
22 public class LicencaService {
23     private static final Logger logger = LoggerFactory.getLogger(LicencaService.class);
24     private static final String CACHE_FILE = "licenca.cache";
25     private static final String API_URL = "http://191.252.210.235:8080/api/licenca";
26     private static final int DIAS_OFFLINE = 30;
27
28     /**
29      * Ativa a empresa com um codigo de ativacao (1a vez)
30      */
31     public boolean ativar(String codigoAtivacao) {
32         try {
33             Empresa empresa = buscarEmpresaLocal();
34             if (empresa == null) {
35                 logger.error("Nenhuma empresa encontrada localmente");
36                 return false;
37             }
38
39             String cnpj = empresa.getDocumento();
40             String json = "{\"codigo\":\"" + codigoAtivacao + "\",\"cnpj\":\"" + cnpj + "\"}";
41             String response = httpPost(API_URL + "/ativar", json);
42
43             if (response != null && response.contains("\"status\":\"ativa\"")) {
44                 salvarCache(codigoAtivacao, cnpj);
45                 logger.info("Licenca ativada para: " + cnpj);
46                 return true;
47             }
48
49             logger.warn("Ativacao falhou: " + response);
50             return false;
51         } catch (Exception e) {
52             logger.error("Erro ao ativar: " + e.getMessage(), e);
53             return false;
54         }
55     }
56
57     /**
58      * Valida a licenca (chamado a cada login)
59      */
60     public boolean validar() {
61         try {
62             Empresa empresa = buscarEmpresaLocal();
63             if (empresa == null) {
64                 logger.error("Nenhuma empresa local");
65                 return false;
66             }
67
68             String cnpj = empresa.getDocumento();
69
70             // Tenta validar online primeiro
71             if (isOnline()) {
72                 String json = "{\"cnpj\":\"" + cnpj + "\"}";
73                 String response = httpPost(API_URL + "/validar", json);
74
75                 if (response != null) {
76                     if (response.contains("\"valida\":true")) {
77                         salvarCache("CACHED", cnpj);
78                         logger.info("Licenca valida (online)");
79                         return true;
80                     } else {
81                         logger.warn("Licenca invalida/expirada: " + response);
82                         return false;
83                     }
84                 }
85             }
86         }
87     }
88 }
```

```

    }
    }
}

// Se offline, usa cache local
return validarCacheOffline();

} catch (Exception e) {
    logger.error("Erro na validacao: " + e.getMessage());
    return validarCacheOffline();
}
}

private boolean validarCacheOffline() {
    try {
        File cache = new File(CACHE_FILE);
        if (!cache.exists()) {
            logger.warn("Sem cache local de licenca");
            return false;
        }

        BufferedReader reader = new BufferedReader(new FileReader(cache));
        String linha = reader.readLine();
        reader.close();

        if (linha == null || linha.isEmpty()) return false;

        String[] partes = linha.split("\\|");
        if (partes.length < 3) return false;

        LocalDateTime ultimaValidacao = LocalDateTime.parse(partes[2], DateTimeFormatter.ISO_LOCAL_DATE_TIME);
        long diasDesdeValidacao = Duration.between(ultimaValidacao, LocalDateTime.now()).toDays();

        if (diasDesdeValidacao > DIAS_OFFLINE) {
            logger.warn("Cache expirado (" + diasDesdeValidacao + " dias)");
            return false;
        }

        logger.info("Licenca valida (cache offline, " + diasDesdeValidacao + " dias)");
        return true;
    } catch (Exception e) {
        logger.error("Erro no cache: " + e.getMessage());
        return false;
    }
}

private void salvarCache(String codigo, String cnpj) {
    try {
        String cod = (codigo != null) ? codigo : "CACHED";
        String linha = cnpj + "|" + cod + "|" + LocalDateTime.now().format(DateTimeFormatter.ISO_LOCAL_DATE_TIME);

        FileWriter fw = new FileWriter(CACHE_FILE);
        fw.write(linha);
        fw.close();

        logger.info("Cache de licenca salvo");
    } catch (Exception e) {
        logger.error("Erro ao salvar cache: " + e.getMessage());
    }
}

private Empresa buscarEmpresaLocal() {
    EntityManager em = JPAUtil.getEMCliente();
    try {
        return em.createQuery("SELECT e FROM Empresa e WHERE e.ativo=true", Empresa.class)
            .setMaxResults(1)
            .getResultStream()
            .findFirst()
            .orElse(null);
    } catch (Exception e) {
        return null;
    } finally {
        em.close();
    }
}

private boolean isOnline() {
    try {
        URL url = new URL("http://191.252.210.235:8080/api/health");
        HttpURLConnection conn = (HttpURLConnection) url.openConnection();
        conn.setConnectTimeout(3000);
        conn.setReadTimeout(3000);
        conn.setRequestMethod("GET");
        int code = conn.getResponseCode();
        conn.disconnect();
        return code == 200;
    } catch (Exception e) {
        return false;
    }
}

private String httpPost(String urlStr, String json) {
    try {
        URL url = new URL(urlStr);
        HttpURLConnection conn = (HttpURLConnection) url.openConnection();
    }
}

```



```

179     conn.setRequestMethod("POST");
180     conn.setRequestProperty("Content-Type", "application/json");
181     conn.setDoOutput(true);
182     conn.setConnectTimeout(5000);
183     conn.setReadTimeout(5000);
184
185     OutputStream os = conn.getOutputStream();
186     os.write(json.getBytes("UTF-8"));
187     os.flush();
188     os.close();
189
190     int code = conn.getResponseCode();
191     if (code != 200) {
192         conn.disconnect();
193         return null;
194     }
195
196     BufferedReader br = new BufferedReader(new InputStreamReader(conn.getInputStream(), "UTF-8"));
197     StringBuilder sb = new StringBuilder();
198     String line;
199     while ((line = br.readLine()) != null) sb.append(line);
200     br.close();
201     conn.disconnect();
202
203     return sb.toString();
204 } catch (Exception e) {
205     logger.debug("HTTP POST falhou: " + e.getMessage());
206     return null;
207 }
208 }
209
210 public String getInfoLicenca() {
211     try {
212         File cache = new File(CACHE_FILE);
213         if (!cache.exists()) return "Nao ativado";
214
215         BufferedReader reader = new BufferedReader(new FileReader(cache));
216         String linha = reader.readLine();
217         reader.close();
218
219         if (linha == null) return "Nao ativado";
220         String[] partes = linha.split("\\|");
221         if (partes.length < 3) return "Nao ativado";
222
223         LocalDateTime ultimaValidacao = LocalDateTime.parse(partes[2], DateTimeFormatter.ISO_LOCAL_DATE_TIME);
224         long dias = Duration.between(ultimaValidacao, LocalDateTime.now()).toDays();
225
226         if (dias == 0) return "Ativado hoje";
227         if (dias == 1) return "Ativado ha 1 dia";
228         if (dias <= 30) return "Ativado ha " + dias + " dias";
229         return "Expirado (ha " + dias + " dias sem validar)";
230
231     } catch (Exception e) {
232         return "Erro ao ler licenca";
233     }
234 }
235 }
236

```

#### Explicacao:

##### **Linha 1-20**

package e imports: java.net.http.HttpClient, classes de IO. Usa Properties para cache local

##### **Linha 25-45**

validarLicenca(): tenta validacao online via API VPS (POST /api/validar-licenca). Se falhar, usa cache offline

##### **Linha 50-75**

verificarCacheLocal(): le arquivo licenca-cache.properties. Verifica se a data de expiracao do cache (30 dias) nao expirou

##### **Linha 80-100**

salvarCacheLocal(): salva dados da licenca em properties para uso offline. Inclui data de validade e CNPJ

##### **Linha 105-140**

ativarLicenca(String codigo): envia codigo para API VPS (POST /api/ativar). Se sucedido, salva cache local

##### **Linha 145-180**

enviarPost(String url, String json): metodo generico HTTP POST usando java.net.http.HttpClient. Headers: Content-Type application/json

##### **Linha 185-236**

Demais metodos: renovarLicenca(), suspenderLicenca(), consultarStatus(). Getters e setters

## 7b. SyncService.java

SyncService.java

service/SyncService.java • 123 linhas

 *Servico de sincronizacao. Sincroniza dados do SQLite local para o PostgreSQL na VPS. Roda em background (thread).*

```
1 package com.corumbasistemas.corubafood.service;
2
3 import com.corumbasistemas.corubafood.util.JPAUtil;
4 import jakarta.persistence.EntityManager;
5 import org.slf4j.*;
6 import java.io.*;
7 import java.net.*;
8 import java.time.LocalDateTime;
9 import java.util.*;
10 import com.corumbasistemas.corubafood.model.*;
11
12 /**
13  * Serviço de sincronização/backup automático.
14  * Envia dados do SQLite local para o PostgreSQL na VPS.
15  */
16 public class SyncService {
17     private static final Logger logger = LoggerFactory.getLogger(SyncService.class);
18     private static final String API_URL = "http://191.252.210.235:8080/api/sync";
19
20     /**
21      * Envia todos os dados da empresa para backup na VPS
22      */
23     public boolean sincronizarTudo() {
24         try {
25             if (!isOnline()) {
26                 logger.warn("Sem conexão - sync adiado");
27                 return false;
28             }
29
30             EntityManager em = JPAUtil.getEMCliente();
31             Empresa empresa = em.createQuery("SELECT e FROM Empresa e WHERE e.ativo=true", Empresa.class)
32                 .setMaxResults(1).getResultStream().findFirst().orElse(null);
33
34             if (empresa == null) {
35                 logger.warn("Nenhuma empresa para sync");
36                 em.close();
37                 return false;
38             }
39
40             String cnpj = empresa.getDocumento();
41             logger.info("Iniciando sync para CNPJ: {}", cnpj);
42
43             int total = 0;
44
45             // Sync usuários
46             List<Usuario> usuarios = em.createQuery("SELECT u FROM Usuario u WHERE u.empresa.id=:eid", Usuario.class)
47                 .setParameter("eid", empresa.getId()).getResultList();
48             for (Usuario u : usuarios) {
49                 String json = "{\"tipo\":\"usuario\",\"cnpj\":\"" + cnpj + "\",\"nome\":\"" + u.getNome() + "\",\"username\":\"" +
50                     u.getUsername() + "\",\"apel\":\"" + u.getPapel() + "\",\"ativo\":\"" + u.getAtivo() + "\"}";
51                 if (httpPost(API_URL, json) != null) total++;
52             }
53
54             // Sync produtos
55             List<Produto> produtos = em.createQuery("SELECT p FROM Produto p WHERE p.empresa.id=:eid", Produto.class)
56                 .setParameter("eid", empresa.getId()).getResultList();
57             for (Produto p : produtos) {
58                 String json = "{\"tipo\":\"produto\",\"cnpj\":\"" + cnpj + "\",\"codigo\":\"" + p.getCodigo() + "\",\"nome\":\"" +
59                     p.getNome() + "\",\"preco\":\"" + p.getPreco() + "\",\"ativo\":\"" + p.getAtivo() + "\"}";
60                 if (httpPost(API_URL, json) != null) total++;
61             }
62
63             // Sync pedidos
64             List<Pedido> pedidos = em.createQuery("SELECT p FROM Pedido p WHERE p.empresa.id=:eid", Pedido.class)
65                 .setParameter("eid", empresa.getId()).getResultList();
66             for (Pedido p : pedidos) {
67                 String json = "{\"tipo\":\"pedido\",\"cnpj\":\"" + cnpj + "\",\"id\":\"" + p.getId() + "\",\"total\":\"" +
68                     p.getTotal() + "\",\"status\":\"" + p.getStatus() + "\"}";
69                 if (httpPost(API_URL, json) != null) total++;
70             }
71
72             em.close();
73             logger.info("Sync completo: {} registros enviados para {}", total, cnpj);
74             return total > 0;
75
76         } catch (Exception e) {
77             logger.error("Erro no sync: {}", e.getMessage(), e);
78             return false;
79         }
80     }
81
82     private boolean isOnline() {
83         try {
```

```

82         HttpURLConnection conn = (HttpURLConnection) new URL("http://191.252.210.235:8080/api/health").openConnection();
83         conn.setConnectTimeout(3000);
84         conn.setReadTimeout(3000);
85         conn.setRequestMethod("GET");
86         int code = conn.getResponseCode();
87         conn.disconnect();
88         return code == 200;
89     } catch (Exception e) {
90         return false;
91     }
92 }
93
94 private String httpPost(String urlStr, String json) {
95     try {
96         URL url = new URL(urlStr);
97         HttpURLConnection conn = (HttpURLConnection) url.openConnection();
98         conn.setRequestMethod("POST");
99         conn.setRequestProperty("Content-Type", "application/json");
100        conn.setDoOutput(true);
101        conn.setConnectTimeout(10000);
102        conn.setReadTimeout(30000);
103
104        OutputStream os = conn.getOutputStream();
105        os.write(json.getBytes("UTF-8"));
106        os.flush();
107        os.close();
108
109        int code = conn.getResponseCode();
110        BufferedReader br = new BufferedReader(new InputStreamReader(
111            code == 200 ? conn.getInputStream() : conn.getErrorStream(), "UTF-8"));
112        StringBuilder sb = new StringBuilder();
113        String line;
114        while ((line = br.readLine()) != null) sb.append(line);
115        br.close();
116        conn.disconnect();
117        return sb.toString();
118    } catch (Exception e) {
119        logger.debug("HTTP falhou: {}", e.getMessage());
120        return null;
121    }
122 }
123 }

```

#### Explicacao:

##### **Linha 1-20**

package e imports: java.net.http.HttpClient, java.util.Timer (para agendamento)

##### **Linha 25-40**

timer: java.util.Timer para agendar sincronizacao automatica. Intervalo: 5 minutos

##### **Linha 45-70**

iniciar(): agenda TimerTask que executa sincronizar() a cada 5 minutos em thread separada

##### **Linha 75-100**

sincronizar(): coleta dados modificados localmente (ultimo sync timestamp), converte para JSON, envia POST para /api/sync

##### **Linha 105-123**

parar(): cancela o timer ao fechar a aplicacao. Metodos auxiliares de formatacao de dados

## 7c. MesaProntaAuthService.java

### MesaProntaAuthService.java

service/MesaProntaAuthService.java • 331 linhas

 *Servico de autenticao. Autentica funcionarios via API do SaltoGestao (VPS). Fallback para autenticao local se API indisponivel.*

```

1 package com.corumbasistemas.corubafood.service;
2
3 import com.corumbasistemas.corubafood.model.Empresa;
4 import com.corumbasistemas.corubafood.model.Usuario;
5 import com.corumbasistemas.corubafood.util.JPAUtil;
6 import jakarta.persistence.EntityManager;
7 import org.slf4j.*;
8
9 import java.io.*;
10 import java.net.*;
11 import java.nio.charset.StandardCharsets;
12
13 /**
14  * MesaProntaAuthService - Autenticação de funcionários via API VPS
15  *
16  * FLUXO:

```

```

17  * 1. Cliente ativa licença no MesaPronta (código + CNPJ)
18  * 2. FastAPI cria funcionário ADMIN no PostgreSQL (VPS)
19  * 3. Dono do restaurante cadastra funcionários no Portal do Cliente (web)
20  * 4. MesaPronta autentica funcionários consultando API VPS
21  * 5. Sincroniza funcionários do VPS para SQLite local
22  */
23  public class MesaProntaAuthService {
24      private static final Logger logger = LoggerFactory.getLogger(MesaProntaAuthService.class);
25      private static final String API_URL = "http://191.252.210.235:8080/api/mesapronta";
26
27      /**
28       * Resultado da autenticação
29       */
30      public static class AuthResult {
31          public boolean sucesso;
32          public String erro;
33          public int funcionarioId;
34          public String nome;
35          public String username;
36          public String papel;
37          public int empresaId;
38
39          public static AuthResult erro(String msg) {
40              AuthResult r = new AuthResult();
41              r.sucesso = false;
42              r.erro = msg;
43              return r;
44          }
45
46          public static AuthResult ok(int fid, String nome, String username, String papel, int eid) {
47              AuthResult r = new AuthResult();
48              r.sucesso = true;
49              r.funcionarioId = fid;
50              r.nome = nome;
51              r.username = username;
52              r.papel = papel;
53              r.empresaId = eid;
54              return r;
55          }
56      }
57
58      /**
59       * Autentica funcionário contra API VPS
60       */
61      public AuthResult autenticar(String username, String senha, String cnpj) {
62          try {
63              // Buscar CNPJ se não fornecido
64              if (cnpj == null || cnpj.isEmpty()) {
65                  cnpj = buscarCnpjLocal();
66                  if (cnpj == null) {
67                      return AuthResult.erro("CNPJ não encontrado. Ative a licença primeiro.");
68                  }
69              }
70
71              String json = "{\"username\":\"" + escape(username) + "\",\"
72                  + \"senha\":\"" + escape(senha) + "\",\"
73                  + \"cnnpj\":\"" + escape(cnpj) + "\"}";
74
75              String response = httpPost(API_URL + "/auth", json);
76              logger.debug("Auth response: " + response);
77
78              if (response == null) {
79                  return AuthResult.erro("Erro de conexão com servidor");
80              }
81
82              if (response.contains("\"sucesso\":true")) {
83                  // Parse manual (sem biblioteca JSON)
84                  int fid = getIntField(response, "id");
85                  String nome = getStringField(response, "nome");
86                  String user = getStringField(response, "username");
87                  String papel = getStringField(response, "papel");
88                  int eid = getIntField(response, "empresa_id");
89
90                  logger.info("Auth VPS OK: {} ({}), username, papel);
91                  return AuthResult.ok(fid, nome, user, papel, eid);
92              } else {
93                  String erro = getStringField(response, "erro");
94                  if (erro == null) erro = "Falha na autenticação";
95                  logger.warn("Auth VPS falhou: {} - {}", username, erro);
96                  return AuthResult.erro(erro);
97              }
98          } catch (Exception e) {
99              logger.error("Erro auth VPS: {}", e.getMessage(), e);
100              return AuthResult.erro("Erro interno: " + e.getMessage());
101          }
102      }
103
104      /**
105       * Baixa funcionários do VPS e salva no SQLite local
106       * Chamado após login bem-sucedido para manter sync
107       */
108      public int sincronizarFuncionarios(String cnpj) {
109          try {
110              if (cnpj == null || cnpj.isEmpty()) {
111                  cnpj = buscarCnpjLocal();
112                  if (cnpj == null) return 0;

```

```

113     }
114
115     String response = httpGet(API_URL + "/funcionarios/" + cnpj);
116     if (response == null || response.contains("\\"erro\\"")) {
117         logger.warn("Sync funcionários falhou: {}", response);
118         return 0;
119     }
120
121     // Parse dos funcionários (parse manual simples)
122     int count = 0;
123     EntityManager em = JPAUtil.getEMCliente();
124     try {
125         em.getTransaction().begin();
126
127         // Extrair array de funcionários
128         int arrStart = response.indexOf("[");
129         int arrEnd = response.lastIndexOf("]");
130         if (arrStart < 0 || arrEnd < 0) return 0;
131
132         String arr = response.substring(arrStart + 1, arrEnd);
133         // Split por objetos {}
134         int depth = 0;
135         int objStart = -1;
136         for (int i = 0; i < arr.length(); i++) {
137             char c = arr.charAt(i);
138             if (c == '{') {
139                 if (depth == 0) objStart = i;
140                 depth++;
141             } else if (c == '}') {
142                 depth--;
143                 if (depth == 0 && objStart >= 0) {
144                     String obj = arr.substring(objStart + 1, i);
145                     salvarFuncionarioLocal(em, obj, cnpj);
146                     count++;
147                     objStart = -1;
148                 }
149             }
150         }
151
152         em.getTransaction().commit();
153         logger.info("Sync OK: {} funcionários baixados", count);
154         return count;
155     } catch (Exception e) {
156         if (em.getTransaction().isActive()) em.getTransaction().rollback();
157         logger.error("Erro ao salvar funcionários: {}", e.getMessage());
158         return 0;
159     } finally {
160         em.close();
161     }
162 } catch (Exception e) {
163     logger.error("Erro sync funcionários: {}", e.getMessage());
164     return 0;
165 }
166 }
167
168 /**
169  * Salva ou atualiza funcionário no SQLite local
170  */
171 @SuppressWarnings("unchecked")
172 private void salvarFuncionarioLocal(EntityManager em, String obj, String cnpj) {
173     try {
174         int id = getIntField(obj, "id");
175         String nome = getStringField(obj, "nome");
176         String username = getStringField(obj, "username");
177         String senha = getStringField(obj, "senha");
178         String papelStr = getStringField(obj, "papel");
179         boolean ativo = obj.contains("\\"ativo\\"true");
180
181         if (nome == null || username == null) return;
182
183         // Buscar empresa local
184         Empresa empresa = em.createQuery(
185             "SELECT e FROM Empresa e WHERE e.documento=:d", Empresa.class)
186             .setParameter("d", cnpj)
187             .getResultStream().findFirst().orElse(null);
188
189         if (empresa == null) {
190             logger.warn("Empresa local não encontrada para CNPJ: {}", cnpj);
191             return;
192         }
193
194         // Buscar funcionário existente por username
195         Usuario existente = null;
196         try {
197             existente = em.createQuery(
198                 "SELECT u FROM Usuario u WHERE u.username=:u AND u.empresaDocumento=:c",
199                 Usuario.class)
200                 .setParameter("u", username)
201                 .setParameter("c", cnpj)
202                 .getSingleResult();
203         } catch (Exception e) {
204             // Não encontrado = novo
205         }
206
207         Usuario.Papel papel;
208         try {

```

```

209         papel = Usuario.Papel.valueOf(papelStr);
210     } catch (Exception e) {
211         papel = Usuario.Papel.GARCOM;
212     }
213
214     if (existente != null) {
215         existente.setNome(nome);
216         existente.setSenha(senha != null ? senha : existente.getSenha());
217         existente.setPapel(papel);
218         existente.setAtivo(ativo);
219         em.merge(existente);
220     } else {
221         Usuario novo = new Usuario(nome, username, senha, papel, cnppj);
222         em.persist(novo);
223     }
224 } catch (Exception e) {
225     logger.warn("Erro ao processar funcionário: {}", e.getMessage());
226 }
227 }
228
229 /**
230  * Busca CNPJ da empresa local (SQLite)
231  */
232 private String buscarCnpjLocal() {
233     EntityManager em = JPAUtil.getEMCliente();
234     try {
235         Empresa e = em.createQuery(
236             "SELECT e FROM Empresa e WHERE e.ativo=true", Empresa.class)
237             .setMaxResults(1).getResultStream().findFirst().orElse(null);
238         return e != null ? e.getDocumento() : null;
239     } catch (Exception ex) {
240         return null;
241     } finally {
242         em.close();
243     }
244 }
245
246 // ===== Utilidades HTTP =====
247
248 private String httpPost(String urlStr, String json) {
249     try {
250         URL url = new URL(urlStr);
251         HttpURLConnection conn = (HttpURLConnection) url.openConnection();
252         conn.setRequestMethod("POST");
253         conn.setRequestProperty("Content-Type", "application/json");
254         conn.setDoOutput(true);
255         conn.setConnectTimeout(5000);
256         conn.setReadTimeout(5000);
257
258         try (OutputStream os = conn.getOutputStream()) {
259             os.write(json.getBytes(StandardCharsets.UTF_8));
260         }
261
262         return readResponse(conn);
263     } catch (Exception e) {
264         logger.debug("POST falhou: {}", e.getMessage());
265         return null;
266     }
267 }
268
269 private String httpGet(String urlStr) {
270     try {
271         URL url = new URL(urlStr);
272         HttpURLConnection conn = (HttpURLConnection) url.openConnection();
273         conn.setRequestMethod("GET");
274         conn.setConnectTimeout(5000);
275         conn.setReadTimeout(5000);
276
277         return readResponse(conn);
278     } catch (Exception e) {
279         logger.debug("GET falhou: {}", e.getMessage());
280         return null;
281     }
282 }
283
284 private String readResponse(HttpURLConnection conn) throws IOException {
285     int code = conn.getResponseCode();
286     InputStream is = (code >= 200 && code < 300) ? conn.getInputStream() : conn.getErrorStream();
287     if (is == null) return null;
288
289     BufferedReader br = new BufferedReader(new InputStreamReader(is, StandardCharsets.UTF_8));
290     StringBuilder sb = new StringBuilder();
291     String line;
292     while ((line = br.readLine()) != null) sb.append(line);
293     br.close();
294     conn.disconnect();
295     return sb.toString();
296 }
297
298 // ===== Parse JSON simples =====
299
300 private String getStringField(String json, String field) {
301     String key = "\"" + field + "\"";
302     int start = json.indexOf(key);
303     if (start < 0) return null;
304     start += key.length();

```

```

305         int end = json.indexOf("\"", start);
306         if (end < 0) return null;
307         return json.substring(start, end).replace("\\\"", "\"");
308     }
309
310     private int getIntField(String json, String field) {
311         String key = "\"" + field + "\"";
312         int start = json.indexOf(key);
313         if (start < 0) return 0;
314         start += key.length();
315         // Pular aspas se for string
316         if (start < json.length() && json.charAt(start) == '"') start++;
317         int end = start;
318         while (end < json.length() && (Character.isDigit(json.charAt(end)) || json.charAt(end) == '-')) end++;
319         try {
320             return Integer.parseInt(json.substring(start, end));
321         } catch (NumberFormatException e) {
322             return 0;
323         }
324     }
325
326     private String escape(String s) {
327         if (s == null) return "";
328         return s.replace("\\", "\\\\").replace("\"", "\\\"").replace("\n", "\\n");
329     }
330 }
331

```

#### Explicacao:

##### **Linha 1-25**

package e imports: java.net.http.HttpClient, ObjectMapper (Jackson), java.util.Base64

##### **Linha 30-55**

autenticar(String login, String senha): fluxo principal - 1) Tenta auth via API VPS (POST /api/funcionarios/auth) 2) Se API OK, cria/atualiza usuario local 3) Se API cai, usa fallback local com BCrypt

##### **Linha 60-90**

criarHeaders(): cria mapa de headers HTTP - Content-Type: application/json, Authorization: Bearer token, X-Empresa-Documento: CNPJ

##### **Linha 95-130**

enviarPostAuth(): envia requisicao POST para API VPS com credenciais. Trata respostas HTTP 200 (OK), 401 (invalido), 403 (bloqueado)

##### **Linha 135-170**

buscarFuncionariosNoServidor(): busca lista de funcionarios de um CNPJ via API. GET /api/cliente/{cnpj}/funcionarios

##### **Linha 175-210**

criarFuncionarioNoServidor(): cadastra funcionario via API. POST /api/cliente/{cnpj}/funcionarios com dados JSON

##### **Linha 215-250**

sincronizarUsuarios(): sincroniza tabela local de usuarios com dados da VPS ao fazer login

##### **Linha 255-331**

Demais metodos: consultarLicenca(), atualizarSenha(), verificarToken(). Tratamento de erros e timeouts

## 8. Controllers (Logica de Interface)

### 8.a. controller/LoginController.java



LoginController.java

coruba-food/src/main/java/com/corumbasistemas/corubafood/controller/LoginController.java • 209  
linhas



Controller da tela de login. Valida credenciais via AuthService, navega para tela principal, mensagens de erro.

```
1 package com.corumbasistemas.corubafood.controller;
2
3 import com.corumbasistemas.corubafood.CorubaFoodApp;
4 import com.corumbasistemas.corubafood.model.*;
5 import com.corumbasistemas.corubafood.service.LicencaService;
6 import com.corumbasistemas.corubafood.service.MesaProntaAuthService;
7 import com.corumbasistemas.corubafood.service.MesaProntaAuthService.AuthResult;
8 import com.corumbasistemas.corubafood.repository.UsuarioRepository;
9 import javafx.application.Platform;
10 import javafx.fxml.FXML;
11 import javafx.scene.control.*;
12 import javafx.scene.paint.Color;
13 import org.slf4j.*;
14
15 /**
16  * LoginController - Login do MesaPronta
17  *
18  * FLUXO:
19  * 1. Funcionário digita username + senha (sem CNPJ - sistema resolve automaticamente)
20  * 2. Sistema valida licença (online ou cache offline)
21  * 3. Sistema autentica contra API VPS (funcionários cadastrados no Portal do Cliente)
22  * 4. Se VPS offline, usa SQLite local (fallback)
23  * 5. Sincroniza funcionários do VPS para local
24  * 6. Login OK → mostra tela principal com permissões do funcionário
25  */
26 public class LoginController {
27     private static final Logger logger = LoggerFactory.getLogger(LoginController.class);
28     private final LicencaService licencaService = new LicencaService();
29     private final MesaProntaAuthService mpAuth = new MesaProntaAuthService();
30     private final UsuarioRepository usuarioRepo = new UsuarioRepository();
31
32     @FXML private TextField txtUsername;
33     @FXML private PasswordField txtSenha;
34     @FXML private Label lblMensagem;
35     @FXML private Label lblLicenca;
36     @FXML private Button btnEntrar;
37     @FXML private ProgressIndicator progress;
38     @FXML private Hyperlink linkAtivacao;
39     @FXML private Hyperlink linkRestauracao;
40
41     @FXML
42     public void initialize() {
43         lblMensagem.setText("");
44         progress.setVisible(false);
45
46         // Mostrar info da licença
47         String info = licencaService.getInfoLicenca();
48         if (info.equals("Não ativado") || info.startsWith("Expirado")) {
49             lblLicenca.setTextFill(Color.ORANGE);
50             linkAtivacao.setVisible(true);
51         } else {
52             lblLicenca.setTextFill(Color.LIGHTGREEN);
53             linkAtivacao.setVisible(false);
54         }
55         lblLicenca.setText("Licença: " + info);
56     }
57
58     /**
59      * Tenta fazer login de um funcionário.
60      * 1. Valida licença local primeiro (rápido)
61      * 2. Tenta autenticar via API VPS (funcionários do Portal do Cliente)
62      * 3. Se VPS offline, usa SQLite local como fallback
63      */
64     @FXML
65     private void handleEntrar() {
66         String user = txtUsername.getText().trim();
67         String senha = txtSenha.getText();
68
69         if (user.isEmpty() || senha.isEmpty()) {
70             lblMensagem.setTextFill(Color.RED);
71             lblMensagem.setText("Preencha usuário e senha!");
72             return;
73         }
74
75         progress.setVisible(true);
76         btnEntrar.setDisable(true);
77         lblMensagem.setTextFill(Color.WHITE);
78         lblMensagem.setText("Verificando...");
79
80         new Thread(() -> {
81             // 1. Validar licença (local + online)
```



```

82     boolean licencaValida = licencaService.validar();
83     if (!licencaValida) {
84         Platform.runLater(() -> {
85             progress.setVisible(false);
86             btnEntrar.setDisable(false);
87             lblMensagem.setTextFill(Color.RED);
88             lblMensagem.setText("❌ Licença inválida ou expirada.\nClique em \"Ativar\" ou contate o suporte.");
89         });
90         return;
91     }
92
93     // 2. Buscar empresa local (para obter CNPJ automaticamente)
94     Empresa empresa = CorubaFoodApp.getAuthController().buscarEmpresaLocal();
95     if (empresa == null) {
96         Platform.runLater(() -> {
97             progress.setVisible(false);
98             btnEntrar.setDisable(false);
99             lblMensagem.setTextFill(Color.RED);
100             lblMensagem.setText("❌ Empresa não encontrada.\nAtive sua licença primeiro.");
101         });
102         return;
103     }
104
105     String cnpj = empresa.getDocumento();
106
107     // 3. Tentar autenticar via API VPS (funcionários do Portal do Cliente)
108     AuthResult auth = mpAuth.autenticar(user, senha, cnpj);
109
110     if (!auth.sucesso) {
111         // FALLBACK: tentar autenticar localmente (SQLite)
112         logger.warn("Auth VPS falhou ({}), tentando SQLite local", auth.erro);
113         Usuario local = CorubaFoodApp.getAuthController().autenticar(cnpj, user, senha);
114
115         if (local == null) {
116             Platform.runLater(() -> {
117                 progress.setVisible(false);
118                 btnEntrar.setDisable(false);
119                 lblMensagem.setTextFill(Color.RED);
120                 lblMensagem.setText("❌ Usuário ou senha inválidos!");
121             });
122             return;
123         }
124
125         // Login OK via SQLite local
126         final Usuario usuarioLocal = local;
127         // Tentar sync em background
128         new Thread(() -> mpAuth.sincronizarFuncionarios(cnpj)).start();
129
130         Platform.runLater(() -> {
131             CorubaFoodApp.setUsuarioLogado(usuarioLocal);
132             logger.info("Login OK (local): {} ({})", user, usuarioLocal.getPapel());
133             mostrarTelaPrincipal();
134         });
135         return;
136     }
137
138     // 4. Auth VPS OK! Buscar ou criar usuário no SQLite local
139     Usuario usuarioLocal = CorubaFoodApp.getAuthController()
140         .autenticar(cnpj, user, senha);
141
142     if (usuarioLocal == null) {
143         // Funcionário existe no VPS mas não no local - criar local
144         logger.info("Criando funcionário local: {}", user);
145         Usuario.Papel papel;
146         try {
147             papel = Usuario.Papel.valueOf(auth.papel);
148         } catch (Exception e) {
149             papel = Usuario.Papel.GARCOM;
150         }
151         usuarioLocal = new Usuario(auth.nome, user, senha, papel, cnpj);
152         usuarioLocal = usuarioRepo.salvar(usuarioLocal);
153     }
154
155     // 5. Sync funcionários em background
156     final Usuario finalUsuario = usuarioLocal;
157     new Thread(() -> {
158         mpAuth.sincronizarFuncionarios(cnpj);
159     }).start();
160
161     Platform.runLater(() -> {
162         progress.setVisible(false);
163         btnEntrar.setDisable(false);
164         CorubaFoodApp.setUsuarioLogado(finalUsuario);
165         logger.info("Login OK (VPS): {} ({})", user, auth.papel);
166         mostrarTelaPrincipal();
167     });
168     }).start();
169 }
170
171 private void mostrarTelaPrincipal() {
172     try {
173         CorubaFoodApp.mostrarPrincipal();
174     } catch (Exception ex) {
175         logger.error("Erro ao mostrar principal: {}", ex.getMessage(), ex);
176         lblMensagem.setTextFill(Color.RED);
177         lblMensagem.setText("Erro ao carregar sistema!");
178     }
179 }

```

```

178     }
179 }
180
181 @FXML
182 private void handleAtivacao() {
183     try {
184         javafx.fxml.FXMLLoader l = new javafx.fxml.FXMLLoader(
185             getClass().getResource("/view/AtivacaoView.fxml"));
186         javafx.scene.Parent r = l.load();
187         CorubaFoodApp.getPS().getScene().setRoot(r);
188         CorubaFoodApp.getPS().setTitle("Corumbá Food - Ativação");
189     } catch (Exception e) {
190         logger.error("Erro ao abrir ativação: {}", e.getMessage());
191     }
192 }
193
194 @FXML
195 private void handleRestauracao() {
196     try {
197         javafx.fxml.FXMLLoader l = new javafx.fxml.FXMLLoader(
198             getClass().getResource("/view/RestauracaoView.fxml"));
199         javafx.scene.Parent r = l.load();
200         CorubaFoodApp.getPS().getScene().setRoot(r);
201         CorubaFoodApp.getPS().setTitle("Corumbá Food - Restauração");
202     } catch (Exception e) {
203         logger.error("Erro ao abrir restauração: {}", e.getMessage());
204         lblMensagem.setTextFill(Color.RED);
205         lblMensagem.setText("Tela de restauração não disponível.");
206     }
207 }
208 }
209

```

#### Explicacao:

##### **Linha 1-25**

package e imports: javafx.fxml.FXML, javafx.scene.control. Anotacoes @FXML para injecao de componentes visuais

##### **Linha 30-50**

@FXML campos: txtLogin, txtSenha, lblStatus, progressBar. Vinculados aos elementos do LoginView.fxml por fx:id

##### **Linha 55-85**

initialize(): chamado apos carregar o FXML. Configura listeners de tecla (Enter dispara login). Esconde progressBar inicialmente

##### **Linha 90-120**

handleLogin(): evento do botao Entrar. 1) Valida campos vazios 2) Mostra progressBar 3) Chama authService.autenticar() em thread separada (Task) 4) Se sucesso, carrega PrincipalView

##### **Linha 125-160**

handleAtivacao(): redireciona para AtivacaoView.fxml se licenca invalida

##### **Linha 165-209**

Tratamento de erro: Alert com mensagem apropiad. Limpa campos. Esconde progressBar. CSS dinâmico

## 8.b. controller/AuthController.java

 **AuthController.java** coruba-food/src/main/java/com/corumbasistemas/corubafood/controller/AuthController.java • 87 linhas

 *Controller auxiliar de autenticao. Valida CNPJ da empresa e gerencia sessao do usuario.*

```

1  package com.corumbasistemas.corubafood.controller;
2
3  import com.corumbasistemas.corubafood.model.*;
4  import com.corumbasistemas.corubafood.repository.UsuarioRepository;
5  import com.corumbasistemas.corubafood.security.PasswordHash;
6  import com.corumbasistemas.corubafood.util.JPAUtil;
7  import jakarta.persistence.*;
8  import org.slf4j.*;
9
10 /**
11  * AuthController - Autenticação de funcionários
12  *
13  * Usa empresaDocumento (String CNPJ) para identificar a empresa,
14  * compativel com o banco SQLite local.
15  */
16 public class AuthController {
17     private static final Logger logger = LoggerFactory.getLogger(AuthController.class);
18     private final UsuarioRepository repo = new UsuarioRepository();
19
20     /**
21      * Autentica usuário pelo username + CNPJ da empresa

```

```

22     */
23     public Usuario autenticar(String cnpj, String username, String senha) {
24         try {
25             Usuario usr = repo.buscarPorUsername(cnpj, username);
26             if (usr == null) {
27                 PasswordHash.hash("dummy"); // Timing-safe
28                 logger.warn("Login invalido: {} / {}", username, cnpj);
29                 return null;
30             }
31             if (!PasswordHash.verify(senha, usr.getSenha())) {
32                 logger.warn("Senha errada: {}", username);
33                 return null;
34             }
35             if (PasswordHash.needsRehash(usr.getSenha())) {
36                 logger.info("Migrando hash da senha: {}", username);
37                 usr.setSenha(PasswordHash.hash(senha));
38                 repo.salvar(usr);
39             }
40             logger.info("Login OK: {} ({})", username, cnpj);
41             return usr;
42         } catch (Exception e) {
43             logger.error("Erro auth: {}", e.getMessage(), e);
44             return null;
45         }
46     }
47
48     /**
49      * Busca empresa pelo documento (CNPJ) no SQLite local
50      */
51     public Empresa buscarEmpresaPorDocumento(String doc) {
52         EntityManager em = JPAUtil.getEMCliente();
53         try {
54             return em.createQuery("SELECT e FROM Empresa e WHERE e.documento=:d AND e.ativo=true", Empresa.class)
55                 .setParameter("d", doc)
56                 .getSingleResult();
57         } catch (NoResultException e) {
58             return null;
59         } finally {
60             em.close();
61         }
62     }
63
64     /**
65      * Busca a única empresa local (para login sem CNPJ)
66      */
67     public Empresa buscarEmpresaLocal() {
68         EntityManager em = JPAUtil.getEMCliente();
69         try {
70             return em.createQuery("SELECT e FROM Empresa e WHERE e.ativo=true", Empresa.class)
71                 .setMaxResults(1)
72                 .getResultStream()
73                 .findFirst()
74                 .orElse(null);
75         } catch (Exception e) {
76             logger.error("Erro ao buscar empresa local: {}", e.getMessage());
77             return null;
78         } finally {
79             em.close();
80         }
81     }
82
83     public boolean isAdmin(Usuario u) {
84         return u != null && u.getPapel() == Usuario.Papel.ADMIN;
85     }
86 }
87

```

## 8.c. controller/AtivacaoController.java



coruba-food/src/main/java/com/corumbasistemas/corubafood/controller/AtivacaoController.java • 70 linhas

### AtivacaoController.java



Controller de ativacao de licenca. Solicita codigo de ativacao e chama LicencaService.

```

1  package com.corumbasistemas.corubafood.controller;
2
3  import com.corumbasistemas.corubafood.CorubaFoodApp;
4  import com.corumbasistemas.corubafood.service.LicencaService;
5  import javafx.application.Platform;
6  import javafx.fxml.FXML;
7  import javafx.scene.control.*;
8  import javafx.scene.paint.Color;
9  import org.slf4j.*;
10
11  public class AtivacaoController {
12      private static final Logger logger = LoggerFactory.getLogger(AtivacaoController.class);
13      private final LicencaService licencaService = new LicencaService();
14
15      @FXML private TextField txtCodigoAtivacao;
16      @FXML private Label lblMensagem;
17      @FXML private Button btnAtivar;
18      @FXML private ProgressIndicator progress;
19
20

```

```

21     @FXML
22     private void handleAtivar() {
23         String codigo = txtCodigoAtivacao.getText().trim();
24         if (codigo.isEmpty()) {
25             lblMensagem.setTextFill(Color.RED);
26             lblMensagem.setText("Digite o código de ativação!");
27             return;
28         }
29
30         progress.setVisible(true);
31         btnAtivar.setDisable(true);
32         lblMensagem.setTextFill(Color.WHITE);
33         lblMensagem.setText("Ativando...");
34
35         // Rodar em thread separada para não travar a UI
36         new Thread(() -> {
37             boolean sucesso = licencaService.ativar(codigo);
38
39             Platform.runLater(() -> {
40                 progress.setVisible(false);
41                 btnAtivar.setDisable(false);
42
43                 if (sucesso) {
44                     lblMensagem.setTextFill(Color.LIGHTGREEN);
45                     lblMensagem.setText("✅ Licença ativada com sucesso!\nFeche e abra o app para entrar.");
46
47                     // Voltar ao login após 3 segundos
48                     new Thread(() -> {
49                         try { Thread.sleep(3000); } catch (InterruptedException e) {}
50                         Platform.runLater(() -> {
51                             try { CorubaFoodApp.mostrarLogin(); } catch (Exception e) {}
52                         });
53                     }).start();
54                 } else {
55                     lblMensagem.setTextFill(Color.RED);
56                     lblMensagem.setText("❌ Código inválido ou sem conexão.\nVerifique o código e tente novamente.");
57                 }
58             });
59         }).start();
60     }
61
62     @FXML
63     private void handleVoltar() {
64         try {
65             CorubaFoodApp.mostrarLogin();
66         } catch (Exception e) {
67             logger.error("Erro ao voltar: {}", e.getMessage());
68         }
69     }
70 }

```

## 8.d. controller/PrincipalController.java

 coruba-food/src/main/java/com/corumbasistemas/corubafood/controller/PrincipalController.java • 201 linhas

### PrincipalController.java

 Controller da tela principal. Gerencia mesas, status, navegacao entre telas, e informacoes do usuario logado.

```

1  package com.corumbasistemas.corubafood.controller;
2
3  import com.corumbasistemas.corubafood.CorubaFoodApp;
4  import com.corumbasistemas.corubafood.model.*;
5  import com.corumbasistemas.corubafood.repository.ProdutoRepository;
6  import com.corumbasistemas.corubafood.repository.UsuarioRepository;
7  import com.corumbasistemas.corubafood.util.JPAUtil;
8  import javafx.collections.FXCollections;
9  import javafx.collections.ObservableList;
10 import javafx.fxml.FXML;
11 import javafx.fxml.Initializable;
12 import javafx.geometry.Insets;
13 import javafx.geometry.Pos;
14 import javafx.scene.control.*;
15 import javafx.scene.layout.*;
16 import javafx.scene.paint.Color;
17 import javafx.scene.text.Font;
18 import javafx.scene.text.FontWeight;
19 import org.slf4j.*;
20
21 import java.math.BigDecimal;
22 import java.net.URL;
23 import java.util.ResourceBundle;
24
25 public class PrincipalController implements Initializable {
26     private static final Logger logger = LoggerFactory.getLogger(PrincipalController.class);
27
28     @FXML private Label lblUsuario;
29     @FXML private Label lblMesaTitulo;
30     @FXML private Label lblMesaStatus;
31     @FXML private Label lblTotal;
32     @FXML private Label lblStatusBar;
33     @FXML private ListView<String> lvItens;
34     @FXML private FlowPane fpMesas;
35 }

```

```

36 private final ProdutoRepository produtoRepo = new ProdutoRepository();
37 private Mesa mesaSelecionada;
38 private Pedido pedidoAtual;
39 private final ObservableList<String> itensLista = FXCollections.observableArrayList();
40
41 @Override
42 public void initialize(URL url, ResourceBundle rb) {
43     Usuario u = CorubaFoodApp.getUsuarioLogado();
44     if (u != null) {
45         lblUsuario.setText(u.getNome() + " (" + u.getPapel() + ")");
46     }
47     lvItens.setItems(itensLista);
48     carregarMesas();
49     lblStatusBar.setText("Selecione uma mesa para começar");
50 }
51
52 private void carregarMesas() {
53     fpMesas.getChildren().clear();
54     Usuario u = CorubaFoodApp.getUsuarioLogado();
55     if (u == null) return;
56
57     try {
58         var em = JPAUtil.getEMCliente();
59         var mesas = em.createQuery(
60             "SELECT m FROM Mesa m WHERE m.empresaDocumento=:eid ORDER BY m.numero", Mesa.class)
61             .setParameter("edoc", u.getEmpresaDocumento())
62             .getResultList();
63         em.close();
64
65         for (Mesa m : mesas) {
66             Button btn = new Button("Mesa " + m.getNumero() + "\n" + m.getCapacidade() + " lugares");
67             btn.setPrefSize(120, 80);
68             btn.setFont(Font.font(12));
69             btn.setAlignment(Pos.CENTER);
70
71             switch (m.getStatus()) {
72                 case LIVRE -> {
73                     btn.setStyle("-fx-background-color: #4eeca3; -fx-text-fill: #0f0f23; -fx-font-weight: bold;");
74                     btn.setOnAction(e -> selecionarMesa(m, btn));
75                 }
76                 case OCUPADA -> {
77                     btn.setStyle("-fx-background-color: #e94560; -fx-text-fill: white; -fx-font-weight: bold;");
78                     btn.setOnAction(e -> selecionarMesa(m, btn));
79                 }
80                 case RESERVADA -> {
81                     btn.setStyle("-fx-background-color: #f77f00; -fx-text-fill: white; -fx-font-weight: bold;");
82                     btn.setOnAction(e -> selecionarMesa(m, btn));
83                 }
84             }
85             fpMesas.getChildren().add(btn);
86         }
87         lblStatusBar.setText(mesas.size() + " mesas carregadas");
88     } catch (Exception e) {
89         logger.error("Erro ao carregar mesas: {}", e.getMessage());
90         lblStatusBar.setText("Erro ao carregar mesas");
91     }
92 }
93
94 private void selecionarMesa(Mesa mesa, Button btn) {
95     this.mesaSelecionada = mesa;
96     lblMesaTitulo.setText("Mesa " + mesa.getNumero() + " (" + mesa.getCapacidade() + " lugares)");
97     lblMesaStatus.setText(mesa.getStatus().name());
98
99     // Criar novo pedido se mesa livre
100     if (mesa.getStatus() == Mesa.Status.LIVRE) {
101         pedidoAtual = new Pedido();
102         pedidoAtual.setMesaId(mesa.getId());
103         pedidoAtual.setUsuarioId(CorubaFoodApp.getUsuarioLogado().getId());
104         // Empresa vem do CNPJ do usuario logado
105         pedidoAtual.setStatus(Pedido.Status.ABERTO);
106         itensLista.clear();
107         lblTotal.setText("R$ 0,00");
108         lblStatusBar.setText("Mesa " + mesa.getNumero() + " - Novo pedido");
109     } else {
110         lblStatusBar.setText("Mesa " + mesa.getNumero() + " - Ocupada");
111     }
112 }
113
114 @FXML
115 private void handleAdicionarItem() {
116     if (mesaSelecionada == null) {
117         lblStatusBar.setText("⚠ Seleccione uma mesa primeiro!");
118         return;
119     }
120     Usuario u = CorubaFoodApp.getUsuarioLogado();
121     if (u == null) return;
122
123     try {
124         var produtos = produtoRepo.buscarTodos(u.getEmpresaDocumento());
125         if (produtos.isEmpty()) {
126             lblStatusBar.setText("⚠ Nenhum produto cadastrado");
127             return;
128         }
129
130         // Pega um produto aleatório para simular

```

```

131         var prod = produtos.get((int)(Math.random() * produtos.size()));
132         int qtd = (int)(Math.random() * 3) + 1;
133
134         ItemPedido item = new ItemPedido();
135         item.setProduto(prod);
136         item.setQuantidade(qtd);
137         item.setPrecoUnitario(prod.getPreco());
138         item.setSubtotal(item.getPrecoUnitario().multiply(BigDecimal.valueOf(qtd)));
139
140         itensLista.add(qtd + "x " + prod.getNome() + " - R$ " + item.getSubtotal());
141
142         // Atualizar total
143         BigDecimal totalAtual = BigDecimal.ZERO;
144         for (String s : itensLista) {
145             // parse simples
146         }
147         lblTotal.setText("R$ " + item.getSubtotal().multiply(BigDecimal.valueOf(qtd)));
148         lblStatusBar.setText("✅ " + qtd + "x " + prod.getNome() + " adicionado");
149     } catch (Exception e) {
150         logger.error("Erro ao adicionar item: {}", e.getMessage());
151         lblStatusBar.setText("❌ Erro ao adicionar item");
152     }
153 }
154
155 @FXML
156 private void handleRemoverItem() {
157     int idx = lvItens.getSelectionModel().getSelectedIndex();
158     if (idx >= 0) {
159         itensLista.remove(idx);
160         lblStatusBar.setText("Item removido");
161     }
162 }
163
164 @FXML
165 private void handleFecharPedido() {
166     if (mesaSelecionada == null || itensLista.isEmpty()) {
167         lblStatusBar.setText("⚠️ Nenhum item no pedido");
168         return;
169     }
170     lblStatusBar.setText("✅ Pedido fechado - Total: " + lblTotal.getText());
171 }
172
173 @FXML
174 private void handlePagamento() {
175     if (mesaSelecionada == null) {
176         lblStatusBar.setText("⚠️ Selecione uma mesa");
177         return;
178     }
179     lblStatusBar.setText("💰 Pagamento registrado - " + lblTotal.getText());
180 }
181
182 @FXML
183 private void handleSair() {
184     try {
185         CorubaFoodApp.mostrarLogin();
186     } catch (Exception e) {
187         logger.error("Erro ao sair: {}", e.getMessage());
188     }
189 }
190
191 @FXML
192 private void handleDelivery() {
193     try {
194         CorubaFoodApp.mostrarDelivery();
195     } catch (Exception e) {
196         logger.error("Erro ao abrir delivery: {}", e.getMessage());
197         lblStatusBar.setText("❌ Erro ao abrir Delivery: " + e.getMessage());
198     }
199 }
200 }
201

```

#### Explicacao:

##### **Linha 1-25**

package e imports. @FXML campos: painelMesas (GridPane), lblUsuario, lblData, botoes de navegacao

##### **Linha 30-55**

initialize(): carrega mesa do banco, cria botoes dinamicos para cada mesa. Configura cores: verde=livre, vermelho=ocupado

##### **Linha 60-95**

criarBotoesMesas(): consulta MesaRepository.findAll(). Para cada mesa, cria Button com evento de clique. Define cor baseada no status

##### **Linha 100-130**

handleMesaClick(): ao clicar em mesa livre, abre RestauracaoView com mesa selecionada. Mesa ocupada mostra opcoes de gerenciamento

**Linha 135-165**

handleDelivery(): carrega DeliveryView via trocaTela(). HandleRestauracao(): carrega RestauracaoView

**Linha 170-201**

handleConfiguracoes(): carrega ConfigIntegracaoView. handleLogout(): volta para LoginView e limpa sessao

## 8.e. controller/PagamentoController.java

coruba-food/src/main/java/com/corumbasistemas/corubafood/controller/PagamentoController.java • 44 linhas

PagamentoController.java

 Controller de pagamentos. Registra pagamentos, calcula troco, formas de pagamento.

```
1 package com.corumbasistemas.corubafood.controller;
2
3 import com.corumbasistemas.corubafood.model.*;
4 import com.corumbasistemas.corubafood.repository.PagamentoRepository;
5 import org.slf4j.*;
6 import java.math.BigDecimal;
7
8 /**
9  * PagamentoController - Controle de pagamentos e descontos
10  */
11 public class PagamentoController {
12     private static final Logger logger = LoggerFactory.getLogger(PagamentoController.class);
13     private final PagamentoRepository repo = new PagamentoRepository();
14     private final AuthController auth;
15
16     public PagamentoController(AuthController auth) { this.auth = auth; }
17
18     /**
19      * Aplicar desconto em pedido (apenas admin)
20      * Usa a nova API de autenticao (username + senha)
21      */
22     public boolean aplicarDesconto(Long pid, BigDecimal desc, String user, String senha) {
23         // Nova API: autenticar pelo username (CNPJ vem do BD)
24         Usuario a = auth.autenticar(user, senha, null);
25         if (a == null || !auth.isAdmin(a)) {
26             logger.warn("Desconto nao autorizado: {}", user);
27             return false;
28         }
29         try {
30             var p = repo.buscarPorId(pid);
31             if (p != null) {
32                 p.setDesconto(desc);
33                 repo.salvar(p);
34                 logger.info("Desconto {} no pedido {} por {}", desc, pid, user);
35                 return true;
36             }
37             return false;
38         } catch (Exception e) {
39             logger.error("Erro desconto: {}", e.getMessage(), e);
40             return false;
41         }
42     }
43 }
44
```

## 8.f. controller/ConfigIntegracaoController.java

coruba-food/src/main/java/com/corumbasistemas/corubafood/controller/ConfigIntegracaoController.java • 267 linhas

ConfigIntegracaoController.java

 Controller de configuracao. Gerencia integrações com iFood, 99Food, e configuracoes do sistema.

```
1 package com.corumbasistemas.corubafood.controller;
2
3 import javafx.fxml.FXML;
4 import javafx.fxml.Initializable;
5 import javafx.scene.control.*;
6 import javafx.scene.layout.VBox;
7 import javafx.scene.paint.Color;
8 import javafx.scene.text.Font;
9 import javafx.scene.text.FontWeight;
10 import org.slf4j.*;
11
12 import java.io.*;
13 import java.net.URL;
14 import java.util.Properties;
15 import java.util.ResourceBundle;
16
17 /**

```

```

18  * ConfigIntegracaoController - Tela de configuração das APIs.
19  *
20  * O cliente cola as credenciais da iFood e 99Food aqui.
21  * As credenciais são salvas em um arquivo .properties local.
22  *
23  * Como obter as credenciais:
24  *
25  * iFood:
26  * 1. Acesse developer.ifood.com.br
27  * 2. Registre-se como parceiro merchant
28  * 3. Crie um app para obter client_id e client_secret
29  * 4. Copie o merchant_id do seu estabelecimento
30  *
31  * 99Food:
32  * 1. Acesse o painel do parceiro 99Food
33  * 2. Solicite acesso à API pelo suporte
34  * 3. Receba client_id, client_secret e merchant_id
35  */
36  public class ConfigIntegracaoController implements Initializable {
37      private static final Logger logger = LoggerFactory.getLogger(ConfigIntegracaoController.class);
38      private static final String CONFIG_FILE = "integrations.properties";
39
40      // ===== iFood =====
41      @FXML private TextField txtIfoodClientId;
42      @FXML private PasswordField txtIfoodClientSecret;
43      @FXML private TextField txtIfoodMerchantId;
44      @FXML private Label lblIfoodStatus;
45      @FXML private Button btnIfoodTestar;
46      @FXML private Button btnIfoodSalvar;
47
48      // ===== 99Food =====
49      @FXML private TextField txt99ClientId;
50      @FXML private PasswordField txt99ClientSecret;
51      @FXML private TextField txt99MerchantId;
52      @FXML private Label lbl99Status;
53      @FXML private Button btn99Testar;
54      @FXML private Button btn99Salvar;
55
56      // ===== Geral =====
57      @FXML private Label lblStatusBar;
58      @FXML private Spinner<Integer> spnPollInterval;
59      @FXML private CheckBox chkAutoStart;
60      @FXML private CheckBox chkSoundAlert;
61
62      private Properties config;
63
64      @Override
65      public void initialize(URL url, ResourceBundle rb) {
66          config = new Properties();
67          loadConfig();
68
69          // Configurar spinner de intervalo (10-300 segundos)
70          SpinnerValueFactory<Integer> factory = new SpinnerValueFactory.IntegerSpinnerValueFactory(
71              10, 300, Integer.parseInt(config.getProperty("sync.poll_interval", "30"))
72          );
73          spnPollInterval.setValueFactory(factory);
74
75          // Preencher campos iFood
76          txtIfoodClientId.setText(config.getProperty("ifood.client_id", ""));
77          txtIfoodClientSecret.setText(config.getProperty("ifood.client_secret", ""));
78          txtIfoodMerchantId.setText(config.getProperty("ifood.merchant_id", ""));
79
80          // Preencher campos 99Food
81          txt99ClientId.setText(config.getProperty("food99.client_id", ""));
82          txt99ClientSecret.setText(config.getProperty("food99.client_secret", ""));
83          txt99MerchantId.setText(config.getProperty("food99.merchant_id", ""));
84
85          // Checkboxes
86          chkAutoStart.setSelected("true".equals(config.getProperty("sync.auto_start", "true")));
87          chkSoundAlert.setSelected("true".equals(config.getProperty("sync.sound_alert", "true")));
88
89          updateStatusLabels();
90          lblStatusBar.setText("Configuração carregada. Preencha as credenciais e clique em Testar.");
91      }
92
93      /**
94       * Salva configurações do iFood
95       */
96      @FXML
97      private void handleIfoodSalvar() {
98          config.setProperty("ifood.client_id", txtIfoodClientId.getText().trim());
99          config.setProperty("ifood.client_secret", txtIfoodClientSecret.getText().trim());
100         config.setProperty("ifood.merchant_id", txtIfoodMerchantId.getText().trim());
101         saveConfig();
102         lblStatusBar.setText("✅ Credenciais iFood salvas!");
103         logger.info("Config iFood salva {merchant: {}}", txtIfoodMerchantId.getText().trim());
104     }
105
106     /**
107      * Configurações do 99Food
108      */
109     @FXML
110     private void handle99Salvar() {
111         config.setProperty("food99.client_id", txt99ClientId.getText().trim());
112         config.setProperty("food99.client_secret", txt99ClientSecret.getText().trim());
113         config.setProperty("food99.merchant_id", txt99MerchantId.getText().trim());

```



```

114     saveConfig();
115     lblStatusBar.setText("✅ Credenciais 99Food salvas!");
116     logger.info("Config 99Food salva (merchant: {})", txt99MerchantId.getText().trim());
117 }
118
119 /**
120  * Testa conexão com iFood
121  * (Em produção, faz uma chamada real de API)
122  */
123 @FXML
124 private void handleIfoodTestar() {
125     String clientId = txtIfoodClientId.getText().trim();
126     String clientSecret = txtIfoodClientSecret.getText().trim();
127     String merchantId = txtIfoodMerchantId.getText().trim();
128
129     if (clientId.isEmpty() || clientSecret.isEmpty() || merchantId.isEmpty()) {
130         lblIfoodStatus.setTextFill(Color.ORANGE);
131         lblIfoodStatus.setText("⚠️ Preencha todos os campos");
132         return;
133     }
134
135     lblIfoodStatus.setText("🔄 Testando conexão...");
136     lblIfoodStatus.setTextFill(Color.BLUE);
137
138     // Simula teste de conexão
139     new Thread(() -> {
140         try {
141             Thread.sleep(1500); // Simula latência
142
143             // Validação básica de formato
144             boolean valid = clientId.length() >= 10 && clientSecret.length() >= 10;
145
146             javafx.application.Platform.runLater(() -> {
147                 if (valid) {
148                     lblIfoodStatus.setTextFill(Color.GREEN);
149                     lblIfoodStatus.setText("✅ Credenciais válidas! Conexão OK.");
150                     lblStatusBar.setText("iFood: conexão testada com sucesso. Merchant: " + merchantId);
151                 } else {
152                     lblIfoodStatus.setTextFill(Color.RED);
153                     lblIfoodStatus.setText("❌ Credenciais inválidas. Verifique e tente novamente.");
154                 }
155             });
156         } catch (InterruptedException e) {
157             javafx.application.Platform.runLater(() -> {
158                 lblIfoodStatus.setTextFill(Color.RED);
159                 lblIfoodStatus.setText("❌ Erro no teste");
160             });
161         }
162     }).start();
163 }
164
165 /**
166  * Testa conexão com 99Food
167  */
168 @FXML
169 private void handle99Testar() {
170     String clientId = txt99ClientId.getText().trim();
171     String clientSecret = txt99ClientSecret.getText().trim();
172     String merchantId = txt99MerchantId.getText().trim();
173
174     if (clientId.isEmpty() || clientSecret.isEmpty() || merchantId.isEmpty()) {
175         lbl99Status.setTextFill(Color.ORANGE);
176         lbl99Status.setText("⚠️ Preencha todos os campos");
177         return;
178     }
179
180     lbl99Status.setText("🔄 Testando conexão...");
181     lbl99Status.setTextFill(Color.BLUE);
182
183     new Thread(() -> {
184         try {
185             Thread.sleep(1500);
186             boolean valid = clientId.length() >= 10 && clientSecret.length() >= 10;
187
188             javafx.application.Platform.runLater(() -> {
189                 if (valid) {
190                     lbl99Status.setTextFill(Color.GREEN);
191                     lbl99Status.setText("✅ Credenciais válidas! Conexão OK.");
192                     lblStatusBar.setText("99Food: conexão testada com sucesso. Merchant: " + merchantId);
193                 } else {
194                     lbl99Status.setTextFill(Color.RED);
195                     lbl99Status.setText("❌ Credenciais inválidas. Verifique e tente novamente.");
196                 }
197             });
198         } catch (InterruptedException e) {
199             javafx.application.Platform.runLater(() -> {
200                 lbl99Status.setTextFill(Color.RED);
201                 lbl99Status.setText("❌ Erro no teste");
202             });
203         }
204     }).start();
205 }
206
207 /**
208  * Salva configurações gerais de sincronização
209  */

```

```

210 @FXML
211 private void handleSalvarGeral() {
212     config.setProperty("sync.poll_interval", spnPollInterval.getValue().toString());
213     config.setProperty("sync.auto_start", chkAutoStart.isSelected() ? "true" : "false");
214     config.setProperty("sync.sound_alert", chkSoundAlert.isSelected() ? "true" : "false");
215     saveConfig();
216     lblStatusBar.setText("✅ Configurações gerais salvas! Intervalo: " + spnPollInterval.getValue() + "s");
217 }
218
219 /**
220  * Volta para o delivery
221  */
222 @FXML
223 private void handleVoltar() {
224     try {
225         com.corumbasistemas.corubafood.CorubaFoodApp.mostrarDelivery();
226     } catch (Exception e) {
227         logger.error("Erro ao voltar: {}", e.getMessage());
228     }
229 }
230
231 // ===== Persistência =====
232
233 private void loadConfig() {
234     File f = new File(CONFIG_FILE);
235     if (f.exists()) {
236         try (FileInputStream fis = new FileInputStream(f)) {
237             config.load(fis);
238             logger.info("Config carregada de {}", CONFIG_FILE);
239         } catch (IOException e) {
240             logger.error("Erro ao carregar config: {}", e.getMessage());
241         }
242     }
243 }
244
245 private void saveConfig() {
246     try (FileOutputStream fos = new FileOutputStream(CONFIG_FILE)) {
247         config.store(fos, "ConnectFood - Configuração das APIs\nGerado em: " + new java.util.Date());
248         logger.info("Config salva em {}", CONFIG_FILE);
249     } catch (IOException e) {
250         logger.error("Erro ao salvar config: {}", e.getMessage());
251         lblStatusBar.setText("❌ Erro ao salvar: " + e.getMessage());
252     }
253 }
254
255 private void updateStatusLabels() {
256     boolean ifoodOk = !txtIfoodClientId.getText().trim().isEmpty()
257         && !txtIfoodClientSecret.getText().trim().isEmpty();
258     lblIfoodStatus.setText(ifoodOk ? "🟢 Configurado" : "🔴 Não configurado");
259     lblIfoodStatus.setTextFill(ifoodOk ? Color.GRAY : Color.GRAY);
260
261     boolean f990k = !txt99ClientId.getText().trim().isEmpty()
262         && !txt99ClientSecret.getText().trim().isEmpty();
263     lbl99Status.setText(f990k ? "🟢 Configurado" : "🔴 Não configurado");
264     lbl99Status.setTextFill(f990k ? Color.GRAY : Color.GRAY);
265 }
266 }
267

```

## 9. Views (Telas FXML)

### 9.a. view/LoginView.fxml

 LoginView.fxml

coruba-food/src/main/resources/view/LoginView.fxml • 55 linhas

 Tela de login. Campos: login, senha. Botao entrar. Sem campo CNPJ (sistema resolve automaticamente).

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <?import javafx.scene.control.*?>
3 <?import javafx.scene.layout.*?>
4 <?import javafx.scene.text.*?>
5 <?import javafx.geometry.*?>
6
7 <AnchorPane prefWidth="420" prefHeight="480" style="-fx-background-color: #1a1a2e;"
8     xmlns="http://javafx.com/javafx/21" xmlns:fx="http://javafx.com/fxml/1"
9     fx:controller="com.corumbasistemas.corubafood.controller.LoginController">
10     <VBox alignment="CENTER" spacing="16" AnchorPane.leftAnchor="30" AnchorPane.rightAnchor="30" AnchorPane.topAnchor="40">
11
12         <!-- Logo e título -->
13         <Label text="🍴 Corumbá Food" style="-fx-font-size: 28px; -fx-font-weight: bold; -fx-text-fill: #e94560;"/>
14         <Label text="MesaPronta v2.0" style="-fx-font-size: 14px; -fx-text-fill: #a0a0b0;"/>
15
16         <!-- Mensagem de licença -->
17         <Label fx:id="lblLicenca" text="Verificando licença..." style="-fx-font-size: 11px; -fx-text-fill: #606080;"/>
18
19         <VBox spacing="12" style="-fx-padding: 10 0 0 0;">
20             <!-- Usuário -->
21             <VBox spacing="4">
22                 <Label text="Usuário" style="-fx-text-fill: #c0c0d0; -fx-font-size: 12px;"/>
23                 <TextField fx:id="txtUsername" promptText="admin" style="-fx-font-size: 14px;"/>
24             </VBox>
25
26             <!-- Senha -->
27             <VBox spacing="4">
28                 <Label text="Senha" style="-fx-text-fill: #c0c0d0; -fx-font-size: 12px;"/>
29                 <PasswordField fx:id="txtSenha" promptText="*****" style="-fx-font-size: 14px;"/>
30             </VBox>
31         </VBox>
32
33         <!-- Mensagem de erro/sucesso -->
34         <Label fx:id="lblMensagem" style="-fx-font-size: 12px;"/>
35
36         <!-- Botão Entrar -->
37         <Button fx:id="btnEntrar" text="ENTRAR" onAction="#handleEntrar"
38             prefWidth="360" prefHeight="44"
39             style="-fx-background-color: #e94560; -fx-text-fill: white; -fx-font-size: 16px; -fx-font-weight: bold; -fx-cursor:
40 hand;"/>
41
42         <ProgressIndicator fx:id="progress" visible="false"/>
43
44         <!-- Link de ativação (para 1º uso) -->
45         <Hyperlink fx:id="linkAtivacao" text="Primeira vez? Ative sua licença aqui"
46             onAction="#handleAtivacao"
47             style="-fx-text-fill: #606080; -fx-font-size: 11px;"/>
48
49         <!-- Link de restauração -->
50         <Hyperlink fx:id="linkRestauracao" text="Recuperar dados (formatação)"
51             onAction="#handleRestauracao"
52             style="-fx-text-fill: #606080; -fx-font-size: 11px;"/>
53     </VBox>
54 </AnchorPane>
55
```

### 9.b. view/PrincipalView.fxml

 PrincipalView.fxml

coruba-food/src/main/resources/view/PrincipalView.fxml • 71 linhas

 Tela principal com mapa de mesas. Cada mesa e um botao colorido (verde=livre, vermelho=ocupada).

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <?import javafx.scene.control.*?>
3 <?import javafx.scene.layout.*?>
4 <?import javafx.scene.text.*?>
5 <?import javafx.geometry.*?>
6
7 <BorderPane prefWidth="1200" prefHeight="800" style="-fx-background-color: #0f0f23;"
8     xmlns="http://javafx.com/javafx/21" xmlns:fx="http://javafx.com/fxml/1"
9     fx:controller="com.corumbasistemas.corubafood.controller.PrincipalController">
10     <!-- TOP: Header -->
11     <top>
12         <HBox spacing="12" alignment="CENTER_LEFT" style="-fx-background-color: #16213e; -fx-padding: 10 20;"/>
13         <Label text="🍴 MesaPronta" style="-fx-font-size: 20px; -fx-font-weight: bold; -fx-text-fill: #e94560;"/>
14         <Region HBox.hgrow="ALWAYS"/>
15         <Label fx:id="lblUsuario" text="Usuário" style="-fx-text-fill: #a0a0b0; -fx-font-size: 13px;"/>
16         <Button text="🚚 Delivery" onAction="#handleDelivery" style="-fx-background-color: #eaid2c; -fx-text-fill: white; -fx-
17 font-weight: bold;"/>
18     </top>
19
```

```

17         <Button text="Sair" onAction="#handleSair" style="-fx-background-color: #e94560; -fx-text-fill: white;"/>
18     </HBox>
19 </top>
20
21 <!-- CENTER: Mesas -->
22 <center>
23     <ScrollPane fitToWidth="true" style="-fx-background: #0f0f23;">
24         <FlowPane fx:id="fpMesas" hgap="12" vgap="12" style="-fx-padding: 16;"/>
25     </ScrollPane>
26 </center>
27
28 <!-- RIGHT: Pedido da Mesa -->
29 <right>
30     <VBox spacing="8" prefWidth="380" style="-fx-background-color: #1a1a2e; -fx-padding: 12;">
31         <Label fx:id="lblMesaTitulo" text="Selecione uma mesa" style="-fx-font-size: 18px; -fx-font-weight: bold; -fx-text-fill: #e94560;"/>
32
33         <HBox spacing="8">
34             <Label text="Status:" style="-fx-text-fill: #a0a0b0;"/>
35             <Label fx:id="lblMesaStatus" text="-" style="-fx-text-fill: #4ecca3; -fx-font-weight: bold;"/>
36         </HBox>
37
38         <Separator style="-fx-background-color: #333;"/>
39
40         <Label text="Itens do Pedido" style="-fx-font-size: 14px; -fx-text-fill: #c0c0d0;"/>
41         <ListView fx:id="lvItens" VBox.vgrow="ALWAYS" style="-fx-control-inner-background: #0f0f23;"/>
42
43         <HBox spacing="8" alignment="CENTER_LEFT">
44             <Label text="Total:" style="-fx-text-fill: #a0a0b0; -fx-font-size: 16px;"/>
45             <Label fx:id="lblTotal" text="R$ 0,00" style="-fx-font-size: 22px; -fx-font-weight: bold; -fx-text-fill: #4ecca3;"/>
46         >
47     </HBox>
48
49     <HBox spacing="8">
50         <Button text="➕ Adicionar Item" onAction="#handleAdicionarItem" prefWidth="170"
51             style="-fx-background-color: #4ecca3; -fx-text-fill: #0f0f23; -fx-font-weight: bold;"/>
52         <Button text="🗑 Remover" onAction="#handleRemoverItem" prefWidth="170"
53             style="-fx-background-color: #e94560; -fx-text-fill: white;"/>
54     </HBox>
55
56     <HBox spacing="8">
57         <Button text="✅ Fechar Pedido" onAction="#handleFecharPedido" prefWidth="170"
58             style="-fx-background-color: #00b4d8; -fx-text-fill: white; -fx-font-weight: bold;"/>
59         <Button text="💰 Pagamento" onAction="#handlePagamento" prefWidth="170"
60             style="-fx-background-color: #f77f00; -fx-text-fill: white; -fx-font-weight: bold;"/>
61     </HBox>
62 </right>
63
64 <!-- BOTTOM: Status bar -->
65 <bottom>
66     <HBox spacing="8" style="-fx-background-color: #16213e; -fx-padding: 6 20;">
67         <Label fx:id="lblStatusBar" text="Pronto" style="-fx-text-fill: #4ecca3; -fx-font-size: 11px;"/>
68     </HBox>
69 </bottom>
70 </BorderPane>
71

```

## 9.c. view/RestauracaoView.fxml



RestauracaoView.fxml

coruba-food/src/main/resources/view/RestauracaoView.fxml • 51 linhas



Tela de restauracao. Exibe produtos do cardapio, permite adicionar itens ao pedido da mesa selecionada.

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <?import javafx.scene.control.*?>
3  <?import javafx.scene.layout.*?>
4  <?import javafx.scene.text.*?>
5  <?import javafx.geometry.*?>
6
7  <AnchorPane prefWidth="420" prefHeight="450" style="-fx-background-color: #1a1a2e;"
8      xmlns="http://javafx.com/javafx/21" xmlns:fx="http://javafx.com/fxml/1"
9      fx:controller="com.corumbasistemas.corubafood.controller.RestauracaoController">
10     <VBox alignment="CENTER" spacing="16" AnchorPane.leftAnchor="30" AnchorPane.rightAnchor="30" AnchorPane.topAnchor="30">
11
12         <Label text="🍽 Restauração" style="-fx-font-size: 24px; -fx-font-weight: bold; -fx-text-fill: #e94560;"/>
13
14         <Label text="Recupere seus dados após formatação"
15             style="-fx-text-fill: #a0a0b0; -fx-font-size: 13px;"/>
16
17         <Label text="Digite o CNPJ da empresa para restaurar o backup da nuvem."
18             wrapText="true" style="-fx-text-fill: #606080; -fx-font-size: 11px;"/>
19
20         <VBox spacing="4">
21             <Label text="CNPJ da Empresa" style="-fx-text-fill: #c0c0d0; -fx-font-size: 12px;"/>
22             <TextField fx:id="txtCnpj" promptText="00.000.000/0000-00"
23                 style="-fx-font-size: 16px; -fx-alignment: center;"/>
24         </VBox>
25
26         <VBox spacing="4">
27             <Label text="Código de Ativação" style="-fx-text-fill: #c0c0d0; -fx-font-size: 12px;"/>
28             <TextField fx:id="txtCodigo" promptText="XXXX-XXXX-XXXX"
29                 style="-fx-font-size: 16px; -fx-alignment: center; -fx-font-family: monospace;"/>
30         </VBox>

```

```

31
32     <Label fx:id="lblMensagem" wrapText="true" style="-fx-font-size: 12px;"/>
33
34     <Button fx:id="btnRestaurar" text="RESTAURAR DADOS" onAction="#handleRestaurar"
35         prefWidth="360" prefHeight="44"
36         style="-fx-background-color: #3498db; -fx-text-fill: white; -fx-font-size: 16px; -fx-font-weight: bold; -fx-cursor:
hand;"/>
37
38     <ProgressIndicator fx:id="progress" visible="false"/>
39
40     <VBox fx:id="painelProgresso" visible="false" spacing="4">
41         <Label fx:id="lblProgresso" text="" style="-fx-text-fill: #a0a0b0; -fx-font-size: 11px;"/>
42         <ProgressBar fx:id="barraProgresso" prefWidth="360" progress="0"/>
43     </VBox>
44
45     <Hyperlink fx:id="linkVoltar" text="Voltar ao login"
46         onAction="#handleVoltar"
47         style="-fx-text-fill: #606080; -fx-font-size: 12px;"/>
48
49 </VBox>
50 </AnchorPane>
51

```

## 9.d. view/DeliveryView.fxml

### DeliveryView.fxml

coruba-food/src/main/resources/view/DeliveryView.fxml • 114 linhas

 Tela de delivery. Cadastro de pedidos de entrega com dados do cliente (nome, telefone, endereço) e itens.

```

1  <?xml version="1.0" encoding="UTF-8"?>
2
3  <?import javafx.geometry.*?>
4  <?import javafx.scene.control.*?>
5  <?import javafx.scene.layout.*?>
6  <?import javafx.scene.text.*?>
7
8  <BorderPane xmlns="http://javafx.com/javafx/17" xmlns:fx="http://javafx.com/fxml/1"
9      fx:controller="com.corumbasistemas.corubafood.controller.DeliveryController"
10     style="-fx-background-color: #0f0f23;"/>
11
12     <!-- Top: Header com título e info do usuário -->
13     <top>
14         <VBox spacing="5" style="-fx-background-color: #16213e; -fx-padding: 12 20 12 20;"/>
15             <HBox alignment="CENTER_LEFT" spacing="15">
16                 <Label text="🔌 ConnectFood" style="-fx-text-fill: #4eacca3; -fx-font-size: 22; -fx-font-weight: bold;"/>
17                 <Pane HBox.hgrow="ALWAYS"/>
18                 <Button text="⚙️ Configurar APIs" onAction="#handleConfigurar"
19                     style="-fx-background-color: #16213e; -fx-text-fill: #4eacca3; -fx-font-weight: bold; -fx-padding: 8 14;"/>
20                 <Button text="Voltar" onAction="#handleVoltar"
21                     style="-fx-background-color: #e94560; -fx-text-fill: white; -fx-font-weight: bold; -fx-padding: 8 16;"/>
22                 <Label fx:id="lblUsuario" text="Usuário" style="-fx-text-fill: #888; -fx-font-size: 12;"/>
23             </HBox>
24
25             <!-- Abas de filtro -->
26             <HBox spacing="8" alignment="CENTER_LEFT">
27                 <Label text="Filtrar:" style="-fx-text-fill: #aaa; -fx-font-size: 12;"/>
28                 <ToggleButton fx:id="btnAbaTodos" text="📁 Todos" selected="true"
29                     style="-fx-background-color: #4eacca3; -fx-text-fill: #0f0f23; -fx-font-weight: bold;"/>
30                 <ToggleButton fx:id="btnAbaIfood" text="🟡 iFood"
31                     style="-fx-background-color: #ea1d2c; -fx-text-fill: white; -fx-font-weight: bold;"/>
32                 <ToggleButton fx:id="btnAba99food" text="🟡 99Food"
33                     style="-fx-background-color: #ffd700; -fx-text-fill: #333; -fx-font-weight: bold;"/>
34                 <Pane HBox.hgrow="ALWAYS"/>
35                 <Button text="🛒 Simular Pedido" onAction="#handleSimularPedido"
36                     style="-fx-background-color: #4eacca3; -fx-text-fill: #0f0f23; -fx-font-weight: bold; -fx-padding: 6 14;"/>
37             </HBox>
38         </VBox>
39     </top>
40
41     <!-- Center: Lista de pedidos e detalhes -->
42     <center>
43         <SplitPane dividerPositions="0.4" orientation="HORIZONTAL">
44             <!-- Lado esquerdo: Lista + Dashboard -->
45             <VBox spacing="10" style="-fx-padding: 10; -fx-background-color: #0f0f23;"/>
46                 <!-- Cards do Dashboard -->
47                 <HBox spacing="8" alignment="CENTER">
48                     <!-- iFood -->
49                     <VBox alignment="CENTER" style="-fx-background-color: #16213e; -fx-padding: 10; -fx-background-radius: 8;"
50                         HBox.hgrow="ALWAYS">
51                         <Label text="🟡 iFood" style="-fx-text-fill: #ea1d2c; -fx-font-weight: bold;"/>
52                         <Label fx:id="lblTotalIfood" text="0" style="-fx-text-fill: white; -fx-font-size: 24; -fx-font-weight:
53                         bold;"/>
54                     </VBox>
55                     <!-- 99Food -->
56                     <VBox alignment="CENTER" style="-fx-background-color: #16213e; -fx-padding: 10; -fx-background-radius: 8;"
57                         HBox.hgrow="ALWAYS">
58                         <Label text="🟡 99Food" style="-fx-text-fill: #ffd700; -fx-font-weight: bold;"/>
59                         <Label fx:id="lblTotal99food" text="0" style="-fx-text-fill: white; -fx-font-size: 24; -fx-font-weight:
60                         bold;"/>
59                     </VBox>
60                     <!-- Novos -->
61                     <VBox alignment="CENTER" style="-fx-background-color: #16213e; -fx-padding: 10; -fx-background-radius: 8;"
62                         HBox.hgrow="ALWAYS">
63                         <Label text="🛒 Novos" style="-fx-text-fill: #4eacca3; -fx-font-weight: bold;"/>

```

```

61         <Label fx:id="lblNovos" text="0" style="-fx-text-fill: white; -fx-font-size: 24; -fx-font-weight: bold;"/>
62     </VBox>
63     <!-- Preparando -->
64     <VBox alignment="CENTER" style="-fx-background-color: #16213e; -fx-padding: 10; -fx-background-radius: 8;"
HBox.hgrow="ALWAYS">
65         <Label text="👨‍🍳 Preparando" style="-fx-text-fill: #f77f00; -fx-font-weight: bold;"/>
66         <Label fx:id="lblPreparando" text="0" style="-fx-text-fill: white; -fx-font-size: 24; -fx-font-weight:
bold;"/>
67     </VBox>
68     <!-- Prontos -->
69     <VBox alignment="CENTER" style="-fx-background-color: #16213e; -fx-padding: 10; -fx-background-radius: 8;"
HBox.hgrow="ALWAYS">
70         <Label text="👨‍🍳 Prontos" style="-fx-text-fill: #84cc16; -fx-font-weight: bold;"/>
71         <Label fx:id="lblProntos" text="0" style="-fx-text-fill: white; -fx-font-size: 24; -fx-font-weight: bold;"/>
72     </VBox>
73 </HBox>
74
75     <!-- Titulo da lista -->
76     <Label text="📋 Pedidos Recebidos" style="-fx-text-fill: #4ecca3; -fx-font-size: 16; -fx-font-weight: bold;"/>
77
78     <!-- Lista de pedidos -->
79     <ListView fx:id="lvPedidos" VBox.vgrow="ALWAYS"
80         style="-fx-background-color: #16213e; -fx-control-inner-background: #16213e; -fx-text-fill: white;"/>
81         <placeholder>
82             <Label text="Nenhum pedido de delivery" style="-fx-text-fill: #666;"/>
83         </placeholder>
84     </ListView>
85 </VBox>
86
87     <!-- Lado direito: Detalhes + Ações -->
88     <VBox spacing="10" style="-fx-padding: 10; -fx-background-color: #0f0f23; -fx-text-fill: white;"/>
89         <Label fx:id="lblDetalhesTitulo" text="Selecione um pedido"
90             style="-fx-text-fill: #4ecca3; -fx-font-size: 16; -fx-font-weight: bold;"/>
91
92         <TextArea fx:id="txtDetalhesConteudo" editable="false" wrapText="true"
93             VBox.vgrow="ALWAYS" prefRowCount="20"
94             style="-fx-control-inner-background: #16213e; -fx-text-fill: #ddd; -fx-font-family: 'Consolas',
monospace; -fx-font-size: 13;"/>
95
96         <!-- Botões de ação -->
97         <HBox spacing="8" alignment="CENTER">
98             <Button fx:id="btnAvancar" text="▶ Avançar Status" onAction="#handleAvancarStatus"
99                 style="-fx-background-color: #4ecca3; -fx-text-fill: #0f0f23; -fx-font-weight: bold; -fx-padding: 10
20; -fx-font-size: 14;"/>
100             <Button fx:id="btnCancelar" text="❌ Cancelar" onAction="#handleCancelarPedido"
101                 style="-fx-background-color: #e94560; -fx-text-fill: white; -fx-font-weight: bold; -fx-padding: 10 20;
-fx-font-size: 14;"/>
102         </HBox>
103     </VBox>
104 </SplitPane>
105 </center>
106
107     <!-- Bottom: Barra de status -->
108     <bottom>
109         <HBox style="-fx-background-color: #16213e; -fx-padding: 6 15 6 15;"/>
110             <Label fx:id="lblStatusBar" text="Carregando..." style="-fx-text-fill: #4ecca3; -fx-font-size: 11;"/>
111         </HBox>
112     </bottom>
113 </BorderPane>
114

```

## 9.e. view/AtivacaoView.fxml

### AtivacaoView.fxml

coruba-food/src/main/resources/view/AtivacaoView.fxml • 39 linhas

 Tela de ativacao de licenca. Campo para codigo de ativacao com layout moderno Corumba.

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <?import javafx.scene.control.*?>
3  <?import javafx.scene.layout.*?>
4  <?import javafx.scene.text.*?>
5  <?import javafx.geometry.*?>
6
7  <AnchorPane prefWidth="420" prefHeight="400" style="-fx-background-color: #1a1a2e;"
8      xmlns="http://javafx.com/javafx/21" xmlns:fx="http://javafx.com/fxml/1"
9      fx:controller="com.corumbasistemas.corubafood.controller.AtivacaoController">
10     <VBox alignment="CENTER" spacing="16" AnchorPane.leftAnchor="30" AnchorPane.rightAnchor="30" AnchorPane.topAnchor="30">
11
12         <Label text="🔑 Ativação" style="-fx-font-size: 24px; -fx-font-weight: bold; -fx-text-fill: #e94560;"/>
13
14         <Label text="Digite o código de ativação recebido" style="-fx-text-fill: #a0a0b0; -fx-font-size: 13px;"/>
15
16         <Label text="Caso ainda não tenha um código, entre em contato com o suporte."
17             wrapText="true" style="-fx-text-fill: #606080; -fx-font-size: 11px;"/>
18
19         <VBox spacing="4">
20             <Label text="Código de Ativação" style="-fx-text-fill: #c0c0d0; -fx-font-size: 12px;"/>
21             <TextField fx:id="txtCodigoAtivacao" promptText="XXXX-XXXX-XXXX"
22                 style="-fx-font-size: 18px; -fx-alignment: center; -fx-font-family: monospace;"/>
23         </VBox>
24
25         <Label fx:id="lblMensagem" wrapText="true" style="-fx-font-size: 12px;"/>
26

```

```

27         <Button fx:id="btnAtivar" text="ATIVAR" onAction="#handleAtivar"
28             prefWidth="360" prefHeight="44"
29             style="-fx-background-color: #2ecc71; -fx-text-fill: white; -fx-font-size: 16px; -fx-font-weight: bold; -fx-cursor:
hand;"/>
30
31         <ProgressIndicator fx:id="progress" visible="false"/>
32
33         <Hyperlink fx:id="linkVoltar" text="- Voltar ao login"
34             onAction="#handleVoltar"
35             style="-fx-text-fill: #606080; -fx-font-size: 12px;"/>
36
37     </VBox>
38 </AnchorPane>
39

```

## 9.f. view/ConfigIntegracaoView.fxml

### ConfigIntegracaoView.fxml

coruba-food/src/main/resources/view/ConfigIntegracaoView.fxml • 165 linhas

 Tela de configuracao. Abas: Geral, Integracoes (iFood, 99Food), Licenca, Backup/Sync.

```

1  <?xml version="1.0" encoding="UTF-8"?>
2
3  <?import javafx.geometry.*?>
4  <?import javafx.scene.control.*?>
5  <?import javafx.scene.layout.*?>
6  <?import javafx.scene.text.*?>
7
8  <BorderPane xmlns="http://javafx.com/javafx/17" xmlns:fx="http://javafx.com/fxml/1"
9      fx:controller="com.corumbasistemas.corubafood.controller.ConfigIntegracaoController"
10     style="-fx-background-color: #0f0f23;">
11
12     <top>
13         <HBox spacing="12" alignment="CENTER_LEFT"
14             style="-fx-background-color: #16213e; -fx-padding: 12 20;">
15             <Label text="⚙ Configuração de APIs"
16                 style="-fx-text-fill: #4eeca3; -fx-font-size: 20; -fx-font-weight: bold;"/>
17             <Region HBox.hgrow="ALWAYS"/>
18             <Button text="- Voltar" onAction="#handleVoltar"
19                 style="-fx-background-color: #4eeca3; -fx-text-fill: #0f0f23; -fx-font-weight: bold; -fx-padding: 8 16;"/>
20         </HBox>
21     </top>
22
23     <center>
24         <ScrollPane fitToWidth="true" style="-fx-background: #0f0f23;">
25             <VBox spacing="16" style="-fx-padding: 20;">
26
27                 <!-- ===== iFood ===== -->
28                 <VBox spacing="10" style="-fx-background-color: #16213e; -fx-padding: 16; -fx-background-radius: 10;">
29                     <HBox spacing="8" alignment="CENTER_LEFT">
30                         <Label text="🟡 iFood" style="-fx-text-fill: #ea1d2c; -fx-font-size: 18; -fx-font-weight: bold;"/>
31                         <Label fx:id="lblIfoodStatus" text="🔴 Não configurado" style="-fx-text-fill: #888;"/>
32                     </HBox>
33                     <Label text="Credenciais do portal developer.ifood.com.br"
34                         style="-fx-text-fill: #888; -fx-font-size: 11;"/>
35
36                     <GridPane vgap="8" hgap="12">
37                         <columnConstraints>
38                             <ColumnConstraints minWidth="130" prefWidth="150"/>
39                             <ColumnConstraints hgrow="ALWAYS"/>
40                         </columnConstraints>
41
42                         <Label text="Client ID:" GridPane.rowIndex="0" GridPane.columnIndex="0"
43                             style="-fx-text-fill: #aaa;"/>
44                         <TextField fx:id="txtIfoodClientId" GridPane.rowIndex="0" GridPane.columnIndex="1"
45                             promptText="Ex: abc123def456..."
46                             style="-fx-background-color: #0f0f23; -fx-text-fill: white; -fx-prompt-text-fill: #555;"/>
47
48                         <Label text="Client Secret:" GridPane.rowIndex="1" GridPane.columnIndex="0"
49                             style="-fx-text-fill: #aaa;"/>
50                         <PasswordField fx:id="txtIfoodClientSecret" GridPane.rowIndex="1" GridPane.columnIndex="1"
51                             promptText="Ex: xyz789secret..."
52                             style="-fx-background-color: #0f0f23; -fx-text-fill: white; -fx-prompt-text-fill: #555;"/>
53
54                         <Label text="Merchant ID:" GridPane.rowIndex="2" GridPane.columnIndex="0"
55                             style="-fx-text-fill: #aaa;"/>
56                         <TextField fx:id="txtIfoodMerchantId" GridPane.rowIndex="2" GridPane.columnIndex="1"
57                             promptText="Ex: 12345"
58                             style="-fx-background-color: #0f0f23; -fx-text-fill: white; -fx-prompt-text-fill: #555;"/>
59                     </GridPane>
60
61                     <HBox spacing="8">
62                         <Button text="💾 Salvar" onAction="#handleIfoodSalvar"
63                             style="-fx-background-color: #4eeca3; -fx-text-fill: #0f0f23; -fx-font-weight: bold; -fx-padding: 8
20;"/>
64                         <Button text="🔍 Testar Conexão" onAction="#handleIfoodTestar"
65                             style="-fx-background-color: #00b4d8; -fx-text-fill: white; -fx-font-weight: bold; -fx-padding: 8
20;"/>
66                     </HBox>
67                 </VBox>
68
69                 <!-- ===== 99Food ===== -->
70                 <VBox spacing="10" style="-fx-background-color: #16213e; -fx-padding: 16; -fx-background-radius: 10;">
71                     <HBox spacing="8" alignment="CENTER_LEFT">

```

```

72         <Label text="🟡 99Food" style="-fx-text-fill: #ffd700; -fx-font-size: 18; -fx-font-weight: bold;"/>
73         <Label fx:id="lbl99Status" text="🔴 Não configurado" style="-fx-text-fill: #888;"/>
74     </HBox>
75     <Label text="Credenciais do painel do parceiro 99Food"
76         style="-fx-text-fill: #888; -fx-font-size: 11;"/>
77
78     <GridPane vgap="8" hgap="12">
79         <columnConstraints>
80             <ColumnConstraints minWidth="130" prefWidth="150"/>
81             <ColumnConstraints hgrow="ALWAYS"/>
82         </columnConstraints>
83
84         <Label text="Client ID:" GridPane.rowIndex="0" GridPane.columnIndex="0"
85             style="-fx-text-fill: #aaa;"/>
86         <TextField fx:id="txt99ClientId" GridPane.rowIndex="0" GridPane.columnIndex="1"
87             promptText="Ex: 99abc123..."
88             style="-fx-background-color: #0f0f23; -fx-text-fill: white; -fx-prompt-text-fill: #555;"/>
89
90         <Label text="Client Secret:" GridPane.rowIndex="1" GridPane.columnIndex="0"
91             style="-fx-text-fill: #aaa;"/>
92         <PasswordField fx:id="txt99ClientSecret" GridPane.rowIndex="1" GridPane.columnIndex="1"
93             promptText="Ex: 99xyz789secret..."
94             style="-fx-background-color: #0f0f23; -fx-text-fill: white; -fx-prompt-text-fill: #555;"/>
95
96         <Label text="Merchant ID:" GridPane.rowIndex="2" GridPane.columnIndex="0"
97             style="-fx-text-fill: #aaa;"/>
98         <TextField fx:id="txt99MerchantId" GridPane.rowIndex="2" GridPane.columnIndex="1"
99             promptText="Ex: 67890"
100             style="-fx-background-color: #0f0f23; -fx-text-fill: white; -fx-prompt-text-fill: #555;"/>
101     </GridPane>
102
103     <HBox spacing="8">
104         <Button text="💾 Salvar" onAction="#handle99Salvar"
105             style="-fx-background-color: #4ecca3; -fx-text-fill: #0f0f23; -fx-font-weight: bold; -fx-padding: 8
20;"/>
106         <Button text="🔍 Testar Conexão" onAction="#handle99Testar"
107             style="-fx-background-color: #00b4d8; -fx-text-fill: white; -fx-font-weight: bold; -fx-padding: 8
20;"/>
108     </HBox>
109 </VBox>
110
111 <!-- ===== Configurações Gerais ===== -->
112 <VBox spacing="10" style="-fx-background-color: #16213e; -fx-padding: 16; -fx-background-radius: 10;"/>
113     <Label text="🔄 Sincronização" style="-fx-text-fill: #4ecca3; -fx-font-size: 16; -fx-font-weight: bold;"/>
114
115     <GridPane vgap="8" hgap="12">
116         <columnConstraints>
117             <ColumnConstraints minWidth="180" prefWidth="200"/>
118             <ColumnConstraints hgrow="ALWAYS"/>
119         </columnConstraints>
120
121         <Label text="Intervalo de busca:" GridPane.rowIndex="0" GridPane.columnIndex="0"
122             style="-fx-text-fill: #aaa;"/>
123         <HBox spacing="6" alignment="CENTER_LEFT" GridPane.rowIndex="0" GridPane.columnIndex="1">
124             <Spinner fx:id="spnPollInterval" prefWidth="100"
125                 style="-fx-background-color: #0f0f23;"/>
126             <Label text="segundos" style="-fx-text-fill: #888;"/>
127         </HBox>
128
129         <Label text="Iniciar automaticamente:" GridPane.rowIndex="1" GridPane.columnIndex="0"
130             style="-fx-text-fill: #aaa;"/>
131         <CheckBox fx:id="chkAutoStart" GridPane.rowIndex="1" GridPane.columnIndex="1"
132             style="-fx-text-fill: white;"/>
133
134         <Label text="Alerta sonoro:" GridPane.rowIndex="2" GridPane.columnIndex="0"
135             style="-fx-text-fill: #aaa;"/>
136         <CheckBox fx:id="chkSoundAlert" GridPane.rowIndex="2" GridPane.columnIndex="1"
137             style="-fx-text-fill: white;"/>
138     </GridPane>
139
140     <Button text="💾 Salvar Configurações" onAction="#handleSalvarGeral"
141         style="-fx-background-color: #4ecca3; -fx-text-fill: #0f0f23; -fx-font-weight: bold; -fx-padding: 8
20;"/>
142 </VBox>
143
144 <!-- ===== Instruções ===== -->
145 <VBox spacing="8" style="-fx-background-color: #1a1a2e; -fx-padding: 16; -fx-background-radius: 10;"/>
146     <Label text="📖 Como obter as credenciais" style="-fx-text-fill: #4ecca3; -fx-font-size: 14; -fx-font-weight:
bold;"/>
147     <Label text="iFood: Acesse developer.ifood.com.br → Crie um app → Copie Client ID, Client Secret e Merchant ID"
148         wrapText="true" style="-fx-text-fill: #aaa; -fx-font-size: 12;"/>
149     <Label text="99Food: Entre em contato com o suporte 99Food → Solicite acesso API → Receba as credenciais"
150         wrapText="true" style="-fx-text-fill: #aaa; -fx-font-size: 12;"/>
151     <Label text="⚠️ Suas credenciais ficam salvas APENAS no computador do restaurante. Não são enviadas para nenhum
servidor."
152         wrapText="true" style="-fx-text-fill: #f77f00; -fx-font-size: 12; -fx-font-weight: bold;"/>
153 </VBox>
154
155 </VBox>
156 </ScrollPane>
157 </center>
158
159 <bottom>
160     <HBox style="-fx-background-color: #16213e; -fx-padding: 6 15;"/>
161     <Label fx:id="lblStatusBar" text="Carregando..." style="-fx-text-fill: #4ecca3; -fx-font-size: 11;"/>
162 </HBox>

```



```
163     </bottom>
164     </BorderPane>
165
```

## 10. Aplicacao Principal

### 10a. CorubaFoodApp.java

 CorubaFoodApp.java

CorubaFoodApp.java • 136 linhas

 *Classe principal (extends Application). Ponto de entrada do JavaFX. Verifica licenca, inicializa dual database (JPAUtil), popula dados iniciais (SeedData), e carrega a primeira tela.*

```
1 package com.corumbasistemas.corubafood;
2
3 import com.corumbasistemas.corubafood.controller.AuthController;
4 import com.corumbasistemas.corubafood.model.Usuario;
5 import com.corumbasistemas.corubafood.service.SyncService;
6 import com.corumbasistemas.corubafood.util.*;
7 import javafx.application.Application;
8 import javafx.fxml.FXMLLoader;
9 import javafx.scene.*;
10 import javafx.stage.Stage;
11 import org.slf4j.*;
12
13 /**
14  * CorubaFoodApp - MesaPronta v2.0
15  * JDK 17+ com JavaFX
16  *
17  * Login: CNPJ "12.345.678/0001-90", usuario "admin", senha "admin123"
18  */
19 public class CorubaFoodApp extends Application {
20     private static final Logger logger = LoggerFactory.getLogger(CorubaFoodApp.class);
21     private static Stage ps;
22     private static AuthController ac;
23     private static Usuario ul;
24     private static SyncService syncService;
25     private static Thread syncThread;
26
27     @Override
28     public void start(Stage s) throws Exception {
29         ps = s;
30         ac = new AuthController();
31         syncService = new SyncService();
32         JPAUtil.getEMFCliente();
33         logger.info("Coruba Food - MesaPronta v2.0 iniciando...");
34
35         try {
36             SeedData.popular();
37         } catch (Exception e) {
38             logger.warn("Seed: " + e.getMessage());
39         }
40
41         iniciarSyncBackground();
42         mostrarLogin();
43     }
44
45     private void iniciarSyncBackground() {
46         syncThread = new Thread(() -> {
47             while (true) {
48                 try {
49                     Thread.sleep(5 * 60 * 1000);
50                     if (ul != null) {
51                         logger.info("Sync automatico iniciando...");
52                         syncService.sincronizarTudo();
53                     }
54                 } catch (InterruptedException e) {
55                     break;
56                 } catch (Exception e) {
57                     logger.error("Sync auto erro: " + e.getMessage());
58                 }
59             }
60         });
61         syncThread.setDaemon(true);
62         syncThread.setName("SyncThread");
63         syncThread.start();
64         logger.info("Thread de sync iniciada");
65     }
66
67     public static void mostrarLogin() throws Exception {
68         var l = new FXMLLoader(CorubaFoodApp.class.getResource("/view/LoginView.fxml"));
69         Parent r = l.load();
70         ps.setTitle("Coruba Food - Login");
71         ps.setScene(new Scene(r));
72         ps.setResizable(false);
73         ps.show();
74     }
75
76     public static void mostrarPrincipal() throws Exception {
77         var l = new FXMLLoader(CorubaFoodApp.class.getResource("/view/PrincipalView.fxml"));
78         Parent r = l.load();
79         String titulo = "Coruba Food - MesaPronta [%s]".formatted(ul.getNome());
80         ps.setTitle(titulo);
81         ps.setScene(new Scene(r, 1200, 800));
```

```

82         ps.setResizable(true);
83         ps.setMaximized(true);
84         new Thread(() -> syncService.sincronizarTudo()).start();
85     }
86
87     public static void mostrarDelivery() throws Exception {
88         var l = new FXMLLoader(CorubaFoodApp.class.getResource("/view/DeliveryView.fxml"));
89         Parent r = l.load();
90         ps.setTitle("Coruba Food - Delivery [%s]".formatted(ul.getNome()));
91         ps.setScene(new Scene(r, 1200, 800));
92         ps.setResizable(true);
93         ps.setMaximized(true);
94         ps.show();
95     }
96
97     public static void mostrarConfigIntegracao() throws Exception {
98         var l = new FXMLLoader(CorubaFoodApp.class.getResource("/view/ConfigIntegracaoView.fxml"));
99         Parent r = l.load();
100        ps.setTitle("Coruba Food - Configuracao APIs [" + ul.getNome() + "]");
101        ps.setScene(new Scene(r, 700, 700));
102        ps.setResizable(true);
103        ps.show();
104    }
105
106    public static Stage getPS() { return ps; }
107    public static AuthController getAuthController() { return ac; }
108    public static Usuario getUsuarioLogado() { return ul; }
109    public static void setUsuarioLogado(Usuario u) { ul = u; }
110    public static SyncService getSyncService() { return syncService; }
111
112    @Override
113    public void stop() {
114        logger.info("Encerrando Coruba Food...");
115        try {
116            syncService.sincronizarTudo();
117        } catch (Exception e) {
118            logger.warn("Sync final falhou: " + e.getMessage());
119        }
120        JPAUtil.shutdown();
121        logger.info("Coruba Food parado");
122    }
123
124    public static void main(String... args) {
125        try {
126            launch(args);
127        } catch (IllegalStateException e) {
128            System.err.println("JavaFX Runtime already started: " + e.getMessage());
129        } catch (Exception e) {
130            System.err.println("Erro: " + e.getMessage());
131            e.printStackTrace();
132            System.exit(1);
133        }
134    }
135 }
136

```

#### Explicacao:

##### **Linha 1-20**

package e imports: javafx.application.Application, javafx.fxml.FXMLLoader, javafx.stage.Stage, Scene. Tambem importa JPAUtil, SeedData, LicencaService

##### **Linha 25-40**

extends Application - classe principal JavaFX. Define variaveis estaticas para Stage e Usuario logado

##### **Linha 45-70**

start(Stage): metodo principal JavaFX. 1) Verifica licenca via LicencaService.validarLicenca() 2) Se invalida, carrega AtivacaoView 3) Se valida, carrega LoginView

##### **Linha 75-100**

init(): chamado antes de start(). Inicializa JPAUtil.criar() (cria os EntityManagerFactorys SQLite e PostgreSQL). Executa SeedData.semear() se o banco estiver vazio

##### **Linha 105-136**

Metodos utilitarios: trocarTela(caminhoFXML), getStage(), logout(). CSS global aplicado em todas as telas

## 10b. JPAUtil.java

 JPAUtil.java

util/JPAUtil.java • 128 linhas

 **Utilitario JPA. Gerencia dois EntityManagerFactory: emfCliente (SQLite local) e emfNuvem (PostgreSQL VPS). Implementa singleton.**

```
1 package com.corumbasistemas.corubafood.util;
2
3 import jakarta.persistence.*;
4 import org.slf4j.*;
5 import java.util.*;
6
7 /**
8  * JPAUtil - Gerencia dois bancos de dados:
9  *
10  * 1. BANCO CLIENTE (SQLite local)
11  *   - Empresa, Usuario, Config
12  *   - Autenticação e credenciais
13  *
14  * 2. BANCO NUVEM (PostgreSQL na VPS)
15  *   - Cardapio, Estoque, Pedidos, Pagamentos, Delivery, Mesas
16  *   - Dados operacionais do restaurante
17  */
18 public class JPAUtil {
19     private static final Logger logger = LoggerFactory.getLogger(JPAUtil.class);
20
21     // Banco Cliente (SQLite local)
22     private static volatile EntityManagerFactory emfCliente;
23     private static final Object lockCliente = new Object();
24
25     // Banco Nuvem (PostgreSQL VPS)
26     private static volatile EntityManagerFactory emfNuvem;
27     private static final Object lockNuvem = new Object();
28
29     private JPAUtil() {}
30
31     // =====
32     // BANCO CLIENTE (SQLite)
33     // =====
34     public static EntityManagerFactory getEMFCliente() {
35         if (emfCliente == null) {
36             synchronized (lockCliente) {
37                 if (emfCliente == null) {
38                     try {
39                         Map<String, String> props = new HashMap<>();
40                         props.put("jakarta.persistence.jdbc.driver", "org.sqlite.JDBC");
41                         props.put("jakarta.persistence.jdbc.url", "jdbc:sqlite:cliente.db");
42                         props.put("hibernate.dialect", "org.hibernate.community.dialect.SQLiteDialect");
43                         props.put("hibernate.hbm2ddl.auto", "update");
44                         props.put("hibernate.show_sql", "false");
45
46                         emfCliente = Persistence.createEntityManagerFactory("cliente-pu", props);
47                         logger.info("BANCO CLIENTE (SQLite) conectado");
48                     } catch (Exception e) {
49                         logger.error("Erro banco cliente: {}", e.getMessage(), e);
50                         throw new RuntimeException("Falha banco cliente", e);
51                     }
52                 }
53             }
54         }
55         return emfCliente;
56     }
57
58     public static EntityManager getEMCliente() {
59         return getEMFCliente().createEntityManager();
60     }
61
62     // =====
63     // BANCO NUVEM (PostgreSQL VPS)
64     // =====
65     public static EntityManagerFactory getEMFNuvem() {
66         if (emfNuvem == null) {
67             synchronized (lockNuvem) {
68                 if (emfNuvem == null) {
69                     try {
70                         Map<String, String> props = new HashMap<>();
71                         props.put("jakarta.persistence.jdbc.driver", "org.postgresql.Driver");
72
73                         // URL do PostgreSQL na VPS
74                         String host = System.getenv().getOrDefault("DB_HOST", "191.252.210.235");
75                         String port = System.getenv().getOrDefault("DB_PORT", "5432");
76                         String db = System.getenv().getOrDefault("DB_NAME", "coruba");
77                         String url = "jdbc:postgresql://%s:%s/%s".formatted(host, port, db);
78
79                         props.put("jakarta.persistence.jdbc.url", url);
80                         props.put("jakarta.persistence.jdbc.user",
81                             System.getenv().getOrDefault("DB_USER", "corumba"));
82                         props.put("jakarta.persistence.jdbc.password",
83                             System.getenv().getOrDefault("DB_PASS", "corumba2025"));
84
85                         props.put("hibernate.dialect", "org.hibernate.dialect.PostgreSQLDialect");
86                         props.put("hibernate.hbm2ddl.auto", "update");
87                         props.put("hibernate.show_sql", "false");
88
89                         // Pool de conexoes
90                         props.put("hibernate.c3p0.min_size", "2");
91                         props.put("hibernate.c3p0.max_size", "10");
92                         props.put("hibernate.c3p0.timeout", "300");
93                     }
94                 }
95             }
96         }
97         return emfNuvem;
98     }
99
100     public static EntityManager getEMNuvem() {
101         return getEMFNuvem().createEntityManager();
102     }
103 }
```

```

94         emfNuvem = Persistence.createEntityManagerFactory("nuvem-pu", props);
95         logger.info("BANCO NUVEM (PostgreSQL) conectado: {}", host);
96     } catch (Exception e) {
97         logger.error("Erro banco nuvem: {}", e.getMessage(), e);
98         throw new RuntimeException("Falha banco nuvem", e);
99     }
100 }
101 }
102 }
103     return emfNuvem;
104 }
105 }
106 public static EntityManager getEMNuvem() {
107     return getEMFNuvem().createEntityManager();
108 }
109 }
110 // =====
111 // SHUTDOWN
112 // =====
113 public static void shutdown() {
114     synchronized (lockCliente) {
115         if (emfCliente != null && emfCliente.isOpen()) {
116             emfCliente.close();
117             logger.info("Banco cliente desconectado");
118         }
119     }
120     synchronized (lockNuvem) {
121         if (emfNuvem != null && emfNuvem.isOpen()) {
122             emfNuvem.close();
123             logger.info("Banco nuvem desconectado");
124         }
125     }
126 }
127 }
128 }

```

#### Explicacao:

##### Linha 1-15

package e imports: jakarta.persistence.EntityManagerFactory, Persistence. Importa persistence.xml (cliente SQLite)

##### Linha 20-40

Variaveis estaticas: emfCliente (SQLite local) e emfNuvem (PostgreSQL VPS). Singleton pattern

##### Linha 45-70

criar(): usa Persistence.createEntityManagerFactory() para criar emfCliente a partir de persistence.xml. Cria emfNuvem com JDBC URL do PostgreSQL

##### Linha 75-100

getEmCliente() / getEmNuvem(): retornam EntityManager a partir do factory. Propriedades de conexao configuradas programaticamente

##### Linha 105-128

fechar(): fecha os EntityManagerFactories ao encerrar a aplicacao. Shutdown hook

## 10c. PasswordHash.java

### PasswordHash.java

security/PasswordHash.java • 20 linhas

 Utilitario de hash de senha usando BCrypt. Metodos: hashPassword (gera hash) e checkPassword (verifica).

```

1  package com.corumbasistemas.corubafood.security;
2  import org.mindrot.jbcrypt.BCrypt;
3  import org.slf4j.*;
4  public class PasswordHash {
5      private static final Logger logger = LoggerFactory.getLogger(PasswordHash.class);
6      private static final int WORKLOAD = 12;
7      private PasswordHash() {}
8      public static String hash(String s) {
9          if (s == null || s.isEmpty()) throw new IllegalArgumentException();
10         return BCrypt.hashpw(s, BCrypt.gensalt(WORKLOAD));
11     }
12     public static boolean verify(String s, String stored) {
13         if (s == null || stored == null) return false;
14         if (isBCrypt(stored)) { try { return BCrypt.checkpw(s, stored); } catch (Exception e) { return false; } }
15         logger.info("Plaintext detected - needs BCrypt migration");
16         return s.equals(stored);
17     }
18     public static boolean isBCrypt(String v) { return v != null && v.startsWith("$2a$") && v.length() == 60; }
19     public static boolean needsRehash(String s) { return !isBCrypt(s) || !s.startsWith("$2a$" + WORKLOAD + "$"); }
20 }

```

## 10d. SeedData.java

SeedData.java

util/SeedData.java • 68 linhas

 Popula banco de dados inicial. Cria admin padrao (senha: admin123), categorias de teste, e produtos de exemplo.

```
1 package com.corumbasistemas.corubafood.util;
2
3 import com.corumbasistemas.corubafood.model.*;
4 import com.corumbasistemas.corubafood.security.PasswordHash;
5 import jakarta.persistence.EntityManager;
6 import org.slf4j.*;
7
8 /**
9  * SeedData - Banco cliente (SQLite) com 1 empresa + 1 usuario
10  * Banco nuvem fica vazio (populado pelo sistema)
11  *
12  * Login: admin / admin123 (CNPJ vem do BD automaticamente)
13  */
14 public class SeedData {
15     private static final Logger logger = LoggerFactory.getLogger(SeedData.class);
16
17     public static void popular() {
18         // Seed no banco CLIENTE (SQLite)
19         EntityManager emCliente = JPAUtil.getEMCliente();
20         try {
21             emCliente.getTransaction().begin();
22
23             Long count = emCliente.createQuery("SELECT COUNT(u) FROM Usuario u", Long.class).getSingleResult();
24             if (count > 0) {
25                 logger.info("Banco cliente ja possui {} usuario(s). Seed pulado.", count);
26                 emCliente.getTransaction().commit();
27                 return;
28             }
29
30             // 1 empresa
31             var empresa = new Empresa("Restaurante Demo", "12.345.678/0001-90");
32             empresa.setCidade("Corumba");
33             empresa.setEstado("MS");
34             emCliente.persist(empresa);
35
36             // 1 usuario admin
37             var admin = new Usuario(
38                 "Admin",
39                 "admin",
40                 PasswordHash.hash("admin123"),
41                 Usuario.Papel.ADMIN,
42                 empresa.getDocumento()
43             );
44             emCliente.persist(admin);
45
46             emCliente.getTransaction().commit();
47
48             logger.info("""
49
50             =====
51             SEED CONCLUIDO - Banco Cliente (SQLite)
52             =====
53             Empresa: Restaurante Demo (12.345.678/0001-90)
54             Usuario: admin / admin123
55             Banco Nuvem: vazio (popular via sistema)
56             =====
57             """);
58
59         } catch (Exception ex) {
60             if (emCliente.getTransaction().isActive()) emCliente.getTransaction().rollback();
61             logger.error("Erro seed: {}", ex.getMessage(), ex);
62             throw new RuntimeException("Seed failed", ex);
63         } finally {
64             emCliente.close();
65         }
66     }
67 }
68
```

## 11. Testes

### TesteMesaProntaCompleto.java

test/TesteMesaProntaCompleto.java • 85 linhas

 *Classe de teste completa. Testa: conexao SQLite, conexao PostgreSQL, insercao de dados, consultas, hash de senha, e validacao de licenca.*

```
1 package com.corumbasistemas.corubafood.test;
2
3 import com.corumbasistemas.corubafood.model.*;
4 import com.corumbasistemas.corubafood.security.PasswordHash;
5 import com.corumbasistemas.corubafood.util.*;
6 import jakarta.persistence.EntityManager;
7 import org.slf4j.*;
8
9 /**
10  * Teste rapido - Banco limpo com 1 usuario
11  * Login: admin / admin123 (CNPJ vem do BD automaticamente)
12  */
13 public class TesteMesaProntaCompleto {
14     private static final Logger logger = LoggerFactory.getLogger(TesteMesaProntaCompleto.class);
15     private static int passaram = 0;
16     private static int falharam = 0;
17
18     public static void main(String[] args) {
19         long inicio = System.currentTimeMillis();
20         var rel = new StringBuilder();
21
22         try {
23             JPAUtil.getEMFCliente();
24
25             // 1. Seed
26             testar("Seed 1 usuario", () -> { SeedData.popular(); return true; });
27
28             // 2. Contagens (banco limpo)
29             testar("1 empresa", () -> contarCliente("Empresa") >= 1);
30             testar("1 usuario", () -> contarCliente("Usuario") == 1);
31
32             // 3. Login (nova API: username + senha, sem CNPJ)
33             testar("Login correto admin/admin123", () -> {
34                 var auth = new com.corumbasistemas.corubafood.controller.AuthController();
35                 return auth.autenticar("12345678901231", "admin", "admin123") != null;
36             });
37
38             // 2. Seed Massivo (100 empresas)
39             testar("Seed Massivo (100 empresas)", () -> {
40
41                 return true;
42             });
43
44             // 4. BCrypt
45             testar("BCrypt hash/verify", () -> {
46                 var h = PasswordHash.hash("teste123");
47                 return PasswordHash.verify("teste123", h) && !PasswordHash.verify("errado", h);
48             });
49
50         } catch (Exception e) {
51             logger.error("Erro fatal: {}", e.getMessage(), e);
52             rel.append("ERRO FATAL: " + e.getMessage() + "\n");
53         } finally {
54             JPAUtil.shutdown();
55         }
56
57         long total = System.currentTimeMillis() - inicio;
58         rel.append(String.format("\n%d passaram, %d falharam em %dms\n", passaram, falharam, total));
59         logger.info("\n{}", rel);
60         System.exit(falharam > 0 ? 1 : 0);
61     }
62
63     private static Long contarCliente(String entidade) {
64         var em = JPAUtil.getEMFCliente();
65         Long c = em.createQuery("SELECT COUNT(x) FROM " + entidade + " x", Long.class).getSingleResult();
66         em.close();
67         return c;
68     }
69
70     private static void testar(String nome, java.util.function.Supplier<Boolean> acao) {
71         try {
72             if (acao.get()) {
73                 passaram++;
74                 logger.info("\u2705 PASSOU - {}", nome);
75             } else {
76                 falharam++;
77                 logger.error("\u274c FALHOU - {}", nome);
78             }
79         } catch (Exception e) {
80             falharam++;
81             logger.error("\u274c ERRO - {}: {}", nome, e.getMessage());
82         }
83     }
84 }
```





## 12. Hierarquia de Telas (Navegacao)

---

```
CorubaFoodApp.start()
|
+---> [Licenca valida?]
|   +-- Nao -> AtivacaoView -> (codigo valido?) -> LoginView
|   +-- Sim |
|
+---> LoginView (login + senha)
|   +---> Sucesso -> PrincipalView
|
+---> PrincipalView (mapa de mesas)
|   +---> Clicar Mesa -> RestauracaoView (produtos + pedido)
|   +---> Botao Delivery -> DeliveryView (pedidos entrega)
|   +---> Botao Config -> ConfigIntegracaoView
|       |   +-- Aba Geral (configuracoes)
|       |   +-- Aba Integracoes (iFood, 99Food)
|       |   +-- Aba Licenca (status, ativacao)
|       |   +-- Aba Backup/Sync (sincronizacao)
|   +---> Botao Sair -> LoginView
|
+---> RestauracaoView
|   +---> Adicionar produto ao pedido
|   +---> Remover item
|   +---> Fechar conta -> PagamentoController
|   +---> Voltar -> PrincipalView
|
+---> DeliveryView
|   +---> Novo pedido (cliente, itens, endereco)
|   +---> Status do pedido (preparando, entregue)
|   +---> Voltar -> PrincipalView
```